Network Management and Monitoring

AI5049, Winter Semester 2026/27

12 October 2026

Lucas Immanuel Nickel

About me

Lucas Immanuel Nickel

- Network Operations Engineer at DE-CIX
- Research Associate at Fulda University of Applied Sciences
- M.Sc. Applied Computer Science, research stay at the University of Toronto

What is this course about?

Find faults. Make safe changes. Measure the service.

Use that experience to investigate one network-management question.

Our lens: large IP networks

ISP backbones, transit, peering and IXPs.

We study how operators keep these networks working.

The technical block

Date	Time	Topic
12 October 2026	13:45–17:00	Networking 101
13 October 2026	08:45–15:15	Operations + Automation + Incidents
14 October 2026	08:45–15:15	Monitoring / Observability

Short demos, then time to try it yourself.
Use the university lab PCs with Containerlab preinstalled.

ReefNet, from four sides

1

Understand

Healthy request

2

Operate

One incident

3

Automate

Safe repetition

4

Observe

Earlier evidence

The project workshops

Friday sessions: **13:45–18:45**

Date	Focus
30 October 2026	Research questions and sources
13 November 2026	Experimental design
27 November 2026	Analysis and scientific argument
11 December 2026	Forecasts, control loops and agents
22 January 2027	In-class peer review and revision
5 February 2027	Presentations and reproducibility

Fridays: work on your project

Short input together, then project work and individual support.

Bring your code, data, draft and current blocker.

Your project

One question. Teams of 3–4. A meaningful comparison.

Submit	Requirement
Paper	English, IEEE two-column, six pages including references
Artifact	Reproducible evidence
Declaration	Contributions, outside the page limit

Group choice: **12 Oct, 13:45 → 27 Nov, 23:59**.
Nominate one contact person.

What counts towards your grade?

The **paper and artifact** are assessed together. Normally one group grade.
Substantial, documented contribution differences may lead to individual grades.

Paper area	Weight
Introduction & related work	20%
Approach & methodology	25%
Evaluation & results	30%
Discussion & conclusion	15%
Abstract, writing & reproducibility	10%

Who did what?

Record individual and joint work in DECLARATION.md.

Record	Make it concrete
Paper	Sections, figures and revisions
Artifact	Code, configuration and experiments
Joint work	Shared task and each person’s contribution

What goes with the paper?

The paper makes the argument. The **artifact** lets me check it.

- Code and configurations
- Data and software versions
- Reproduction steps
- Seeds, failed runs and exclusions

Public repository: optional. Access for the examiner: required.

Scientific integrity

Follow the IEEE publishing ethics requirements.

- Cite ideas, methods, data and figures
- Check references against the original
- Report what you actually measured
- Keep counter-evidence
- State limitations and uncertainty

A result against your hypothesis is still a result.

AI use and disclosure

Follow the IEEE policy on AI-generated content.

- Spelling and grammar fixes only: disclosure optional
- Generated or substantially rewritten text: disclose
- Generated figures or code: disclose
- Name the system, affected parts and extent in the **Acknowledgments**

You are responsible for the references, code and technical claims.

Dates and deadlines

Date	Deliverable / session
27 November 2026, 23:59	Group selection → Moodle
11 January 2027, 23:59	Full draft + supporting material → Moodle
22 January 2027, in class	Read, write and hand in the paper review
5 February 2027	Presentations in class (ungraded)
12 February 2027, 23:59	Paper, artifact, declaration → Moodle

Optional peer-review bonus

22 January 2027, in class. No make-up date.

One qualifying review improves the individual grade by **one step**, e.g. 2.3 → 2.0.
Best possible grade: **1.0**. Failing grades remain unchanged.

One page, legibly handwritten, German or English.
Work independently. **No AI or other aids.**

For recognized accommodations, contact the lecturer early.

What makes a qualifying review?

A qualifying review covers four things:

- **Summary:** question and main result
- **Method:** assess a methodological choice
- **Judgment:** strength and limitation with reasons
- **Revision:** propose a feasible improvement

I record the four criteria and my reasons.

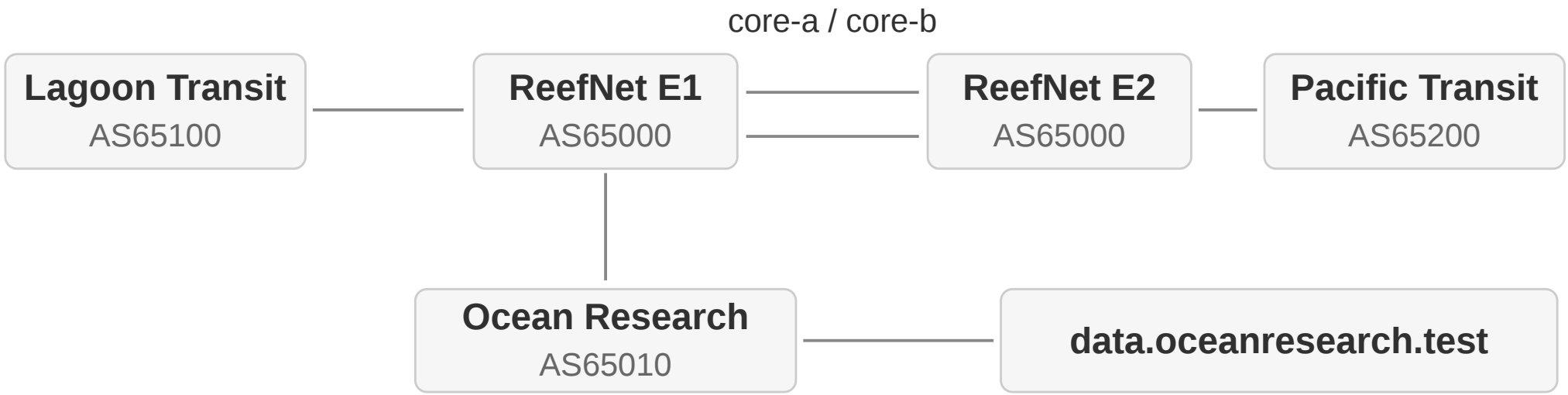
12 October 2026

Networking 101

One network carries the technical block.

Meet ReefNet at BOB1

ReefNet connects **Ocean Research** through two upstreams.



Four views. Four questions.

View	Question
NetBox	What should exist?
Router	What exists now?
Grafana	What changed over time?
External probe	Does the service work?

Follow one service request.

How does this network actually work?

Follow one service request.

In ReefNet: a Lagoon probe requests `data.oceanresearch.test`.

One request crosses several independently operated links.

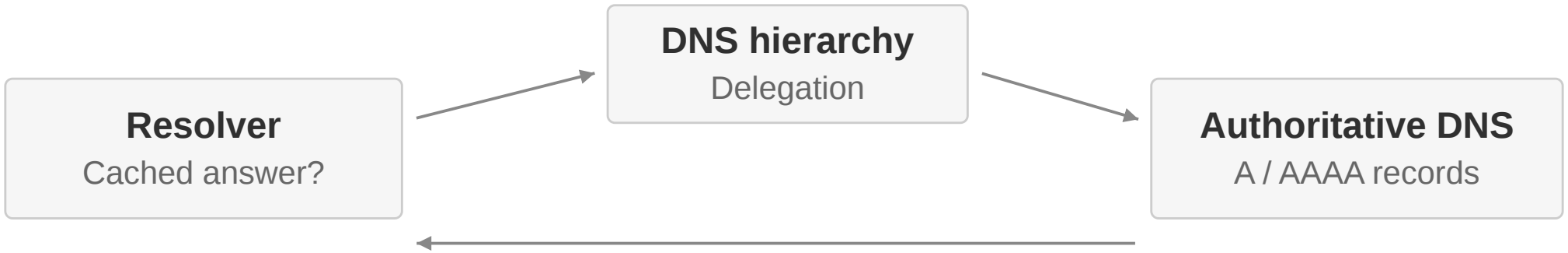


What must work before the application can return a response?

The TCP/IP stack

Application	HTTP, DNS
Transport	TCP, UDP
Internet	IP, ICMP
Link	Ethernet
Physical	Copper, fiber, radio

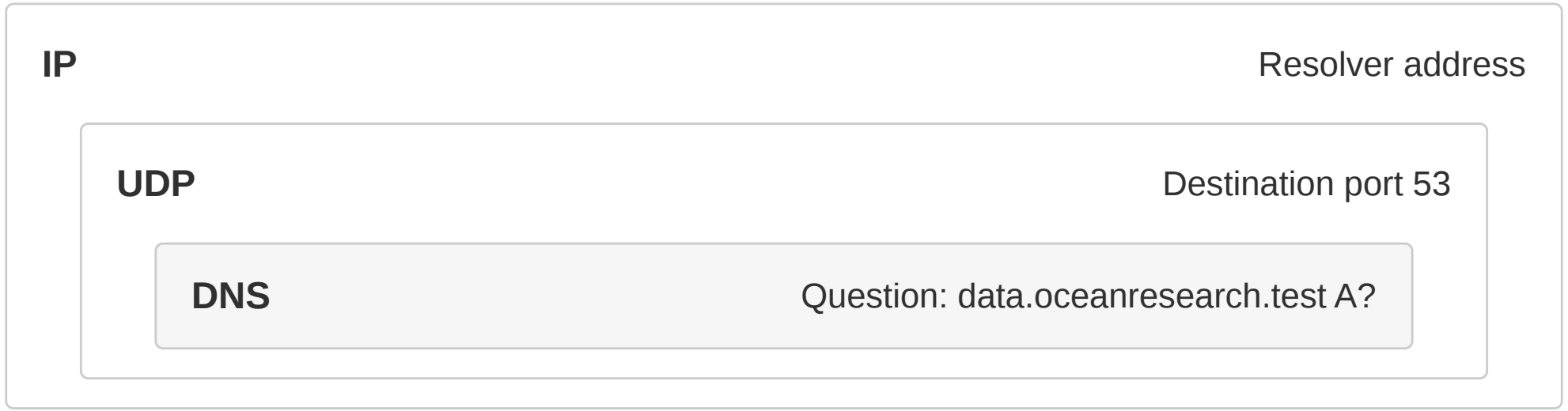
A name needs an address.



An address is an answer about naming, not service health.

A → IPv4
AAAA → IPv6

A DNS query becomes transport data

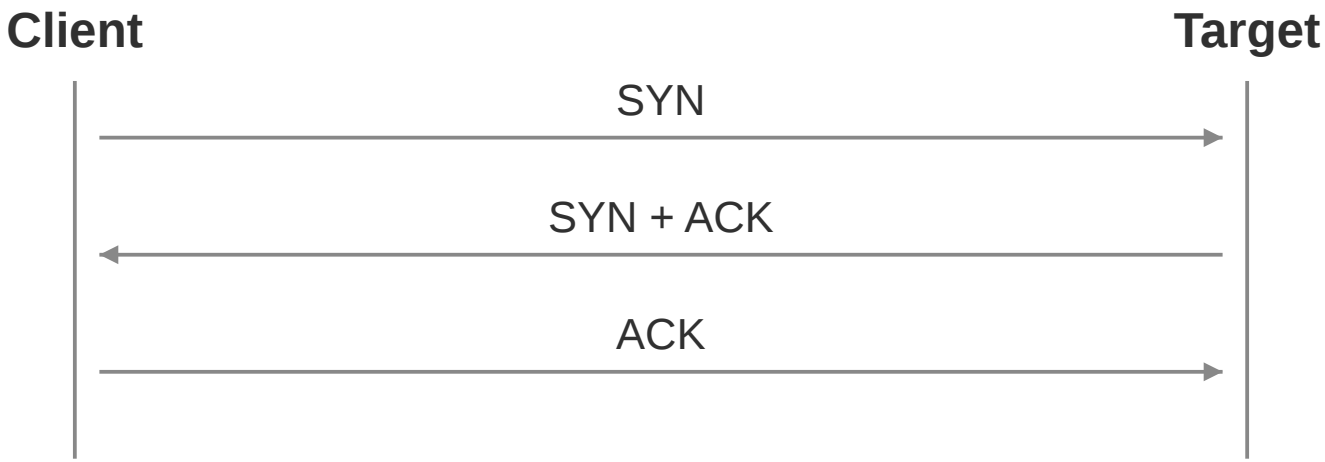


TCP and UDP

	TCP	UDP
Data	Ordered byte stream	Individual datagrams
Connection	Required	None
Retransmission	Built in	Application decides

An IP address + port identifies the transport endpoint.

TCP gets the conversation started.



TCP setup precedes the application response.

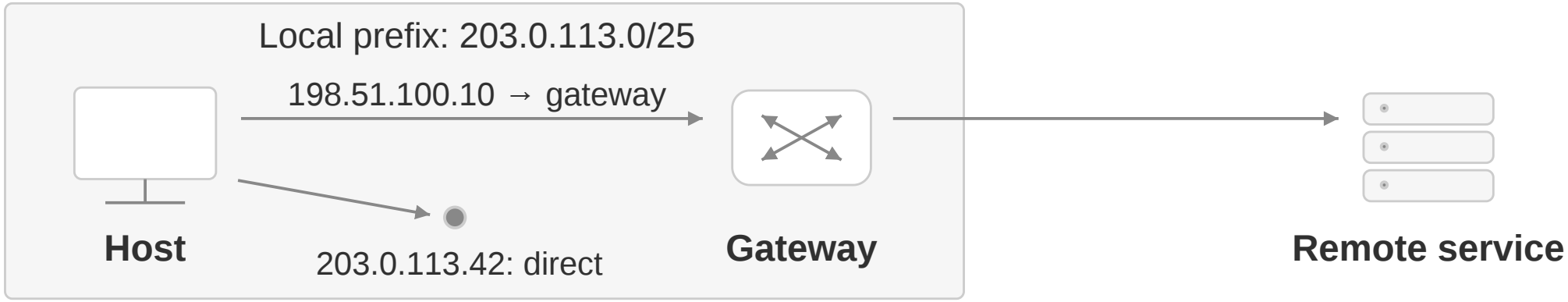
TCP orders bytes, retransmits loss and controls congestion.

IP gives the packet a destination.



Routers use the **destination address** to look up where to forward the packet.

On-link or through a router?



Send directly to an **on-link destination**, otherwise via a **gateway**.

Use the longest matching prefix

Destination: **198.51.100.200**. Example Forwarding Information Base (FIB):

Prefix	Next hop
`0.0.0.0/0`	`192.0.2.1`
`198.51.100.0/24`	`192.0.2.2`
`198.51.100.128/25`	**`192.0.2.3`**

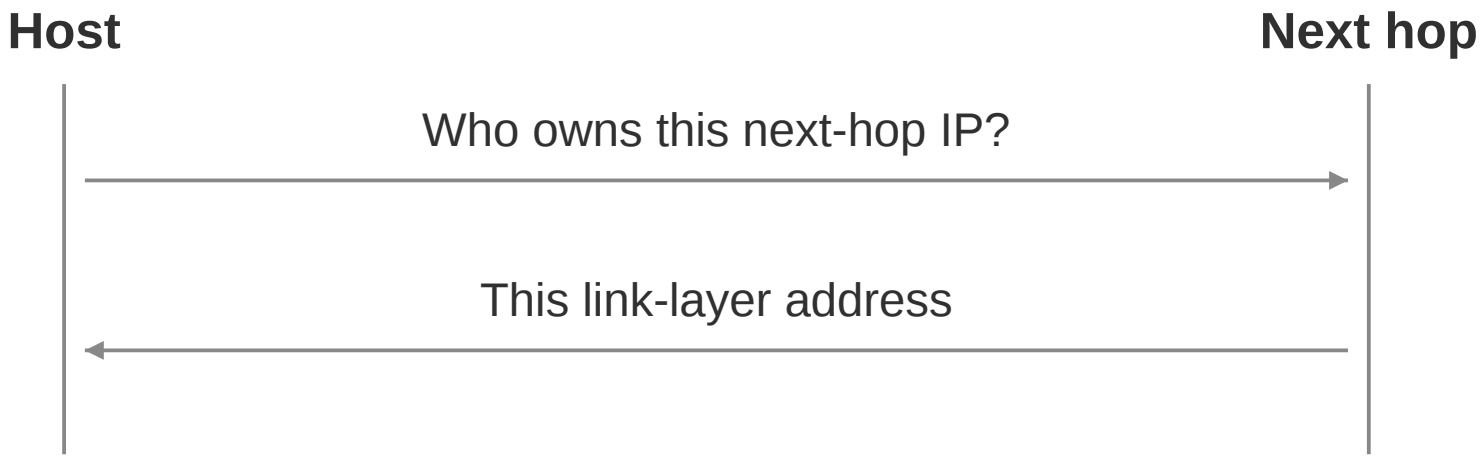
The most specific match wins.

Ethernet carries frames across one link



MAC addresses change per link. The IP packet names the final destination.

Find the next-hop MAC



ARP uses broadcast. ND uses multicast solicitation.

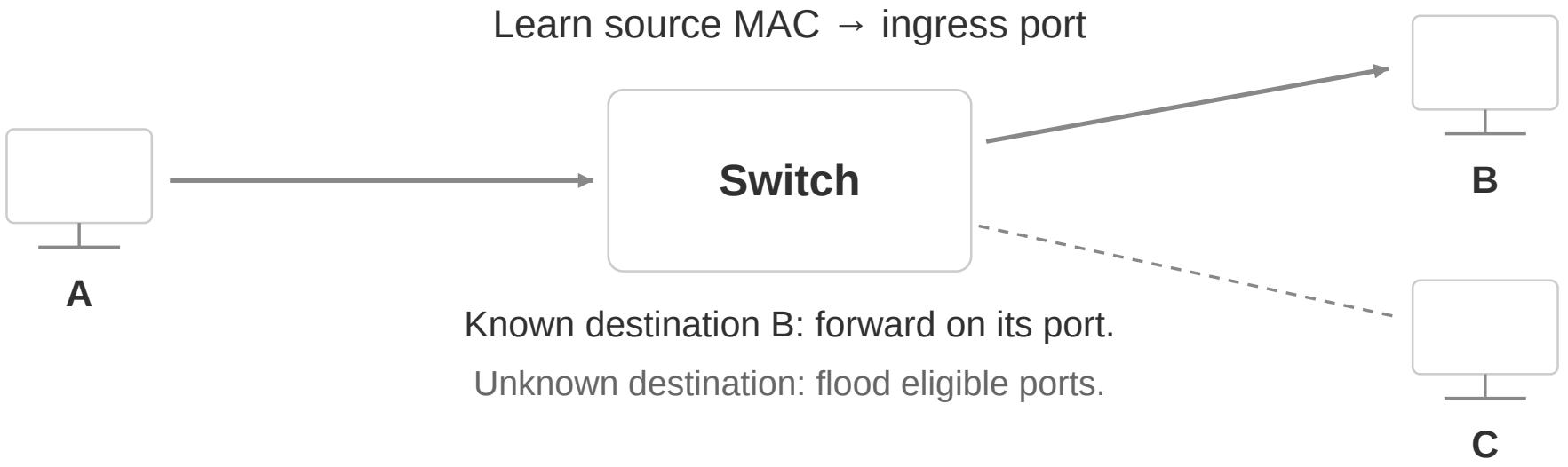
IPv4: ARP

Resolve the selected next hop's IPv4 address to its MAC address.

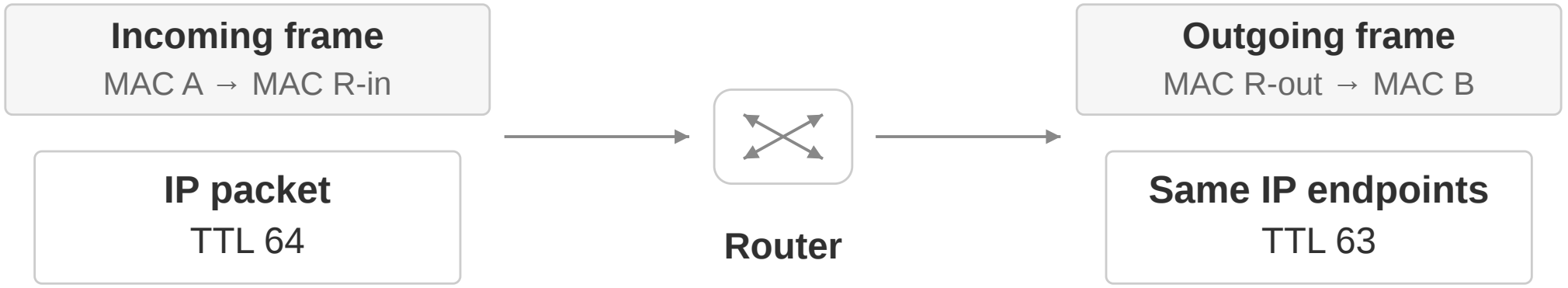
IPv6: Neighbor Discovery

Resolve the selected next hop's IPv6 address using ICMPv6.

Switches learn MAC addresses and forward frames



Routers repeat the process hop by hop



Rebuild the frame. Keep the IP destination. TTL values are illustrative.

The packet keeps its **destination IP**. The link-layer MAC addresses change.

Where do routes come from?

Use the running Networking lab. Leave a Linux node with `exit`, a router with `quit`.

Route source	Origin
Connected	Addressed interface
Static	Operator configuration
Dynamic	Routing protocol

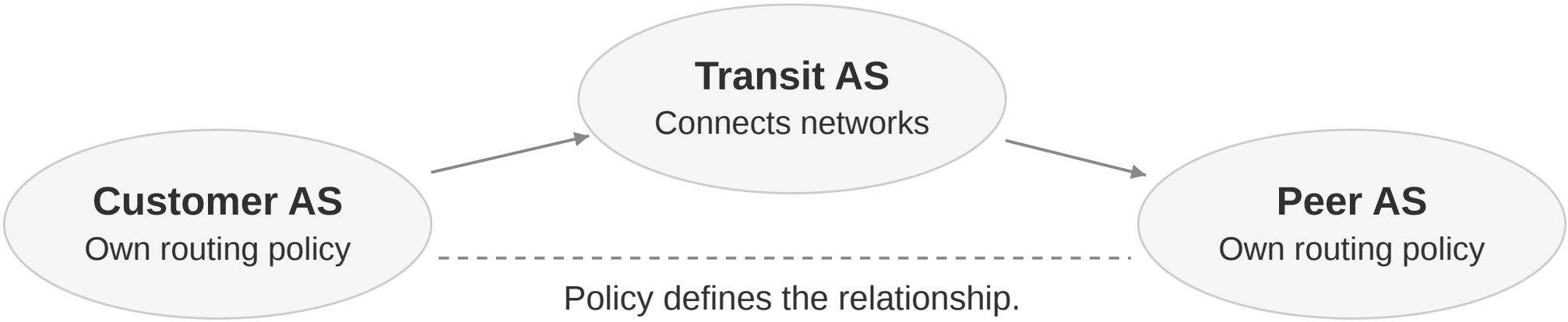
WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default route-table ipv4-unicast summary
```

An AS is an administrative routing domain

An **autonomous system (AS)** presents one routing policy externally.
An **Interior Gateway Protocol (IGP)** finds routes inside it.

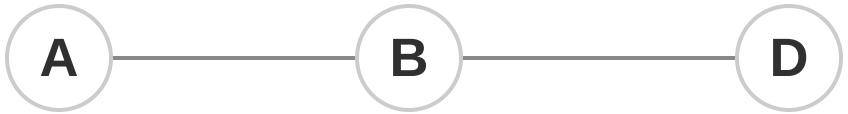


An AS is an administrative boundary, not one router.

Two ways to learn an internal route

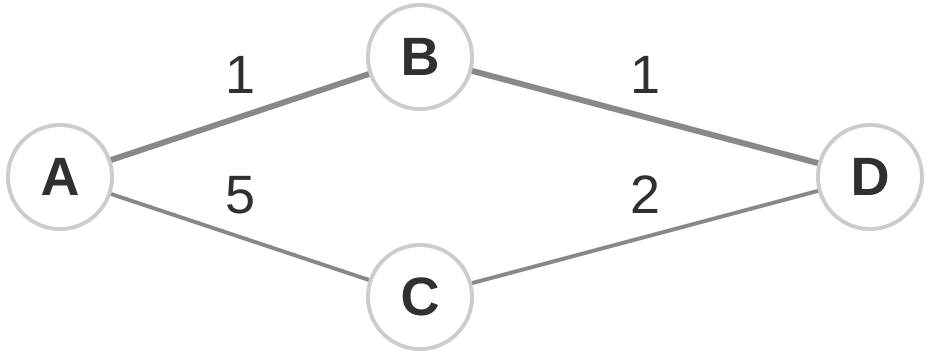
Distance vector (RIP)

B → A: “D is 1 hop away.”



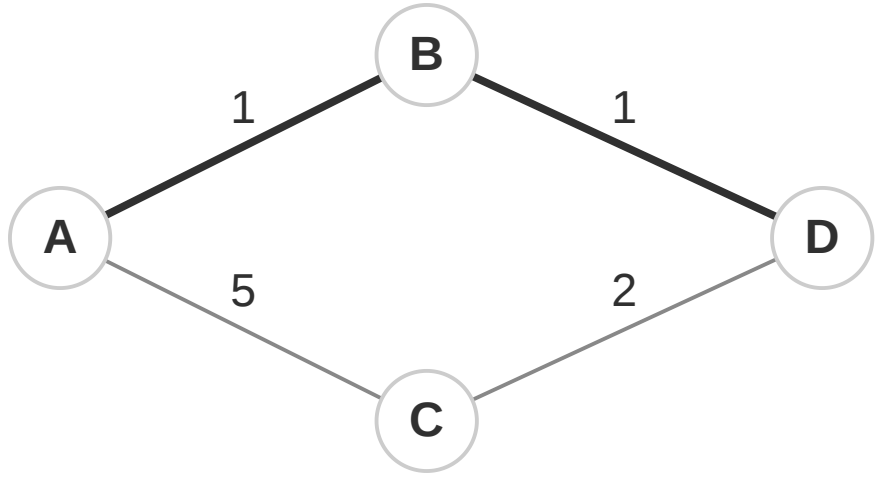
Share distances to destinations.
A adds 1 hop: D via B, 2 hops.
A advertises its own distance onward.

Link state (OSPF)



Share your own links and their costs.
Relay across the area → shared map.
Each router calculates its own paths.

OSPF: each router calculates its own paths



Example costs:
Via B: **1 + 1 = 2**. Via C: 5 + 2 = 7.

ReefNet uses OSPFv2 for IPv4 and OSPFv3 for IPv6.

Both provide paths to the iBGP loopbacks over either core link.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols ospf neighbor
```

BGP exchanges reachability.

Path vector: prefixes, AS paths and other attributes.

eBGP

Routes between ASes.

iBGP

Routes within an AS.
The IGP is still needed.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp neighbor
```

```
show network-instance default protocols bgp routes ipv4 summary
```

BGP separates route families

AFI (Address Family Identifier): IPv4 or IPv6.

SAFI (Subsequent Address Family Identifier): e.g. unicast.

In ReefNet	Routes to destinations such as
`ipv4-unicast`	`198.51.100.0/24`
`ipv6-unicast`	`2001:db8:100::/48`

Established confirms the session is up. Check which families exchange routes.

BGP is policy first.

For comparable routes:

1. Apply local policy.
2. Prefer higher local preference.
3. Compare AS_PATH and further tie-breakers.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp routes ipv4 prefix 198.51.100.0/24
```

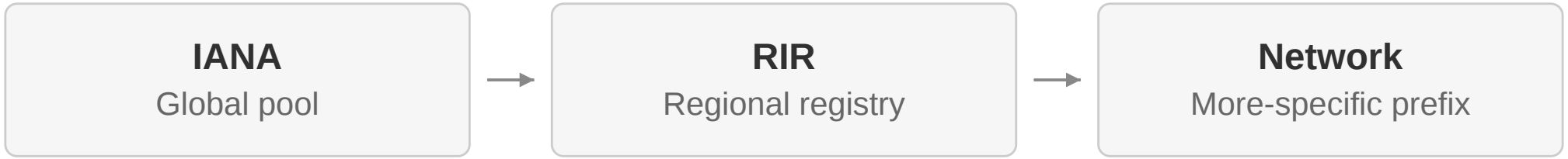
A longer path can be selected on purpose.

Transit and peering

	Transit	Peering
Reachability	Provider supplies broader reachability	Networks exchange selected routes
Commercial relation	Customer pays provider	Often settlement-free

An **IXP** provides shared infrastructure for peering.

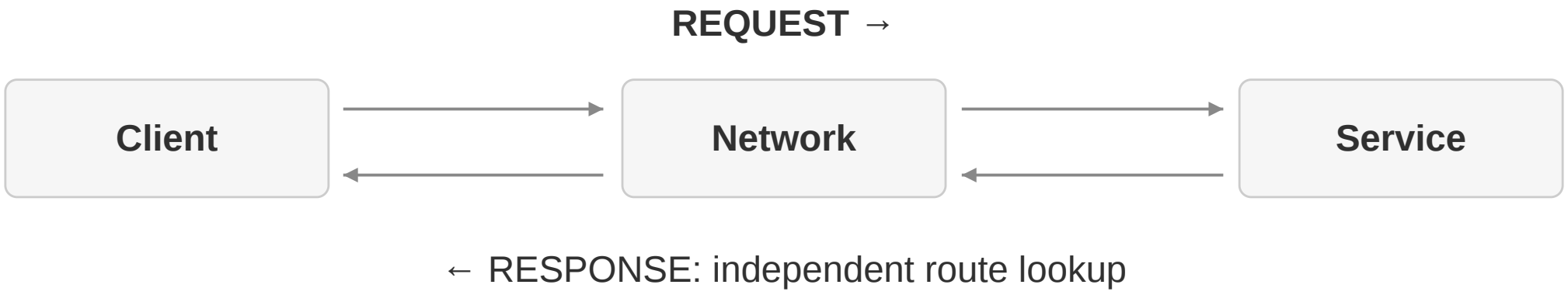
Who assigns the prefixes we route?



Administrative delegation, not a forwarding path

BOB1 uses documentation prefixes.

The response needs its own route.



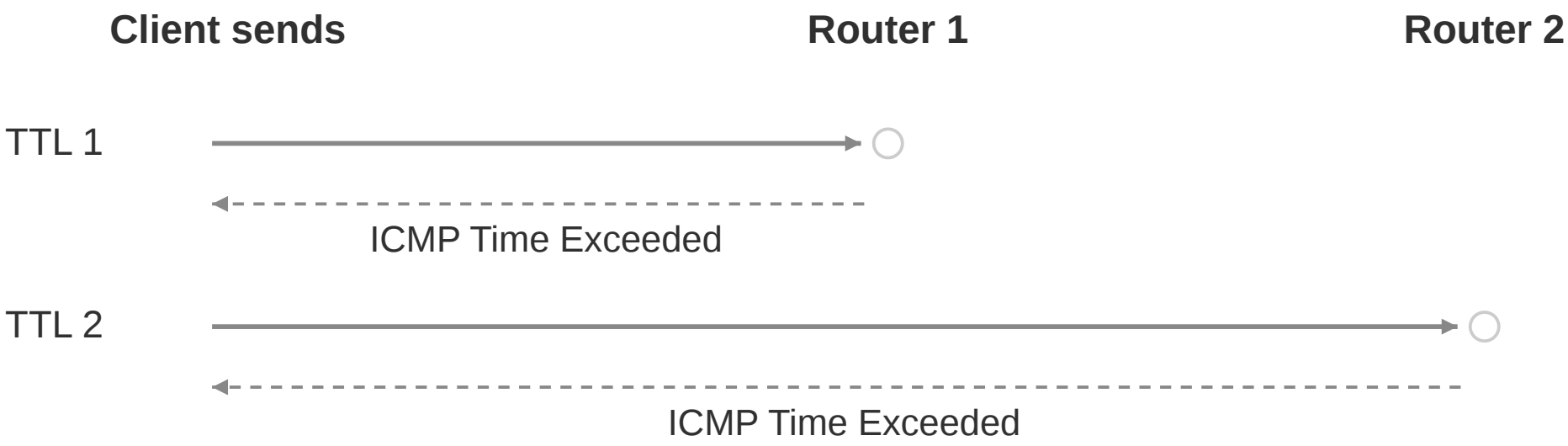
Reachability needs both directions. Paths may be asymmetric.

ICMP: errors and diagnostics

Message	Meaning
Echo Request / Reply	Ping
Destination Unreachable	Cannot deliver
Time Exceeded	TTL / Hop Limit expired
Packet Too Big (IPv6)	Exceeds link MTU

A ping reply proves IP reachability, not application health.

Traceroute expires TTL / Hop Limit.



A missing reply does not identify the failed forwarding hop.

Even a correct path needs healthy links

Medium	Signal	Fault evidence
Copper	Electrical	Link loss, FCS errors
Fiber	Light	Low receive power, FCS errors

Rising **FCS errors** can be evidence even while the link stays up.

Follow the request

How does a request from Lagoon reach `data.oceanresearch.test`,
and how does the reply get back?

60 minutes: 30 for setup, then 30 to explore. In WSL on the lab PC:

```
cd ~;
git clone https://github.com/Stinktopf/network-management-and-monitoring.git;
cd network-management-and-monitoring;
make setup;
make networking;
make task;
```

`make setup` ends with READY FOR CLASS. Check addresses, DNS and HTTP. Sketch the path.
Then try Pacific and IPv6.

Exercise overview

What did your checks tell you?

- Which address came from DNS?
- Which address was your next hop?
- How did you check the return path?
- What did HTTP tell you that ping could not?

Bring one result you can explain and one question that is still open.

13 October 2026

Operations + Automation + Incidents

Yesterday we followed a healthy request through ReefNet.

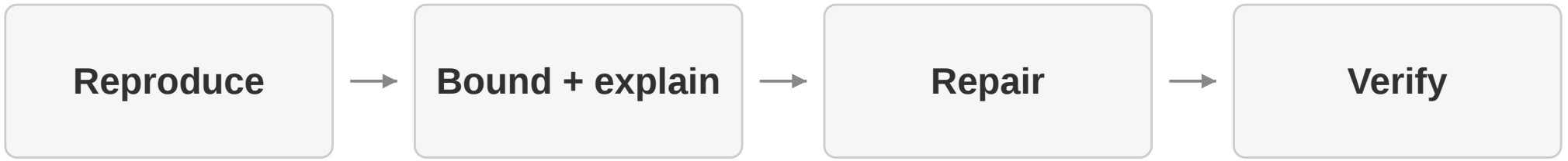
Today Lagoon Transit loses our customer prefix `198.51.100.0/24`.
We repair it in the router CLI, then a second time through gNMI.

At the end of today: **Who Broke the Internet?**

Network Operations

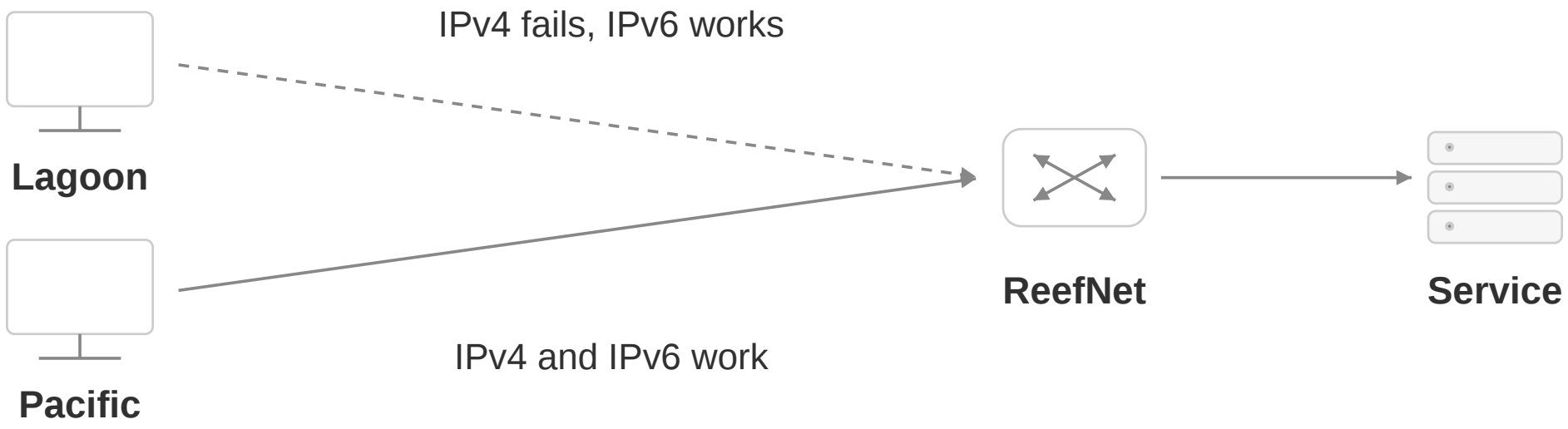
A customer behind Lagoon Transit cannot reach the Ocean Research service.

One incident, from report to recovery



Evidence before action. Recovery before closure.

A customer reports partial reachability



Who cannot reach the service?

WSL `make enter NODE=probe01.bob1.lagoontransit.test`

Inside the Lagoon probe

```
ping -4 -c 3 -W 1 198.51.100.10;
ping -6 -c 3 -W 1 2001:db8:100::10;
```

Use `exit`, then in WSL:

```
make enter NODE=probe01.bob1.pacifictransit.test;
```

Repeat both tests. Which path and address family fail?

Bound the incident

	IPv4	IPv6
Lagoon	FAIL	PASS
Pacific	PASS	PASS

Separate state from service

Management Plane	Configure and monitor	CLI, gNMI, SNMP
Control Plane	Learn and select routes	OSPF, BGP
Data Plane	Forward packets	IP packets, Ethernet frames

Are the BGP sessions established?

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp neighbor
```

IPv4 neighbor	Role
`192.0.2.0`	Ocean Research
`192.0.2.2`	Lagoon Transit
`2001:db8:0:a::`	Lagoon Transit IPv6
`10.255.0.2`	ReefNet edge02 iBGP

Did the customer route arrive?

Compare the customer prefix in both address families.

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default protocols bgp routes ipv4 prefix 198.51.100.0/24
```

```
show network-instance default protocols bgp routes ipv6 prefix 2001:db8:100::/48
```

A route in BGP is not yet proof of forwarding.

Is IPv4 installed for forwarding?

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
show network-instance default route-table ipv4-unicast prefix 198.51.100.0/24
```

Active route → resolved next hop → outgoing interface

Compare the advertisements

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI: toward Lagoon

```
show network-instance default protocols bgp neighbor 192.0.2.2 advertised-routes ipv4
```

Use `quit` before entering the other router.

WSL `make enter NODE=edge02.bob1.reefnet.test`

Router CLI: toward Pacific

```
show network-instance default protocols bgp neighbor 192.0.2.5 advertised-routes ipv4
```

Find `198.51.100.0/24`.

Exercise: find and repair the fault

Users behind Lagoon Transit cannot reach `data.oceanresearch.test`.

In WSL:

```
cd ~/network-management-and-monitoring && git pull;
make operations;
make task;
```

60 minutes: steps 1–5. Find the fault, repair it with the CLI and verify recovery.
Keep your commands and evidence. We compare diagnoses after step 5.

Exercise overview

Which policy does this neighbor use?

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
info from running network-instance default protocols bgp group lagoon-v4
```

```
info from running network-instance default protocols bgp group lagoon-v6
```

What is attached only to IPv4?

Search the whole running config

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI: `flat` prints one full path per line, `grep` filters

```
info flat from running | grep export-policy
```

```
set / network-instance default protocols bgp export-policy [ EXPORT-BGP ]
set / network-instance default protocols bgp group lagoon-v4 export-policy [ BLOCK-CUSTOMER-V4 ]
```

Follow the policy in the running config

Inside edge01, shortened output:

```
info from running network-instance default protocols bgp group lagoon-v4
    export-policy [ BLOCK-CUSTOMER-V4 ]
info from running routing-policy policy BLOCK-CUSTOMER-V4
    statement 10: prefix-set CUSTOMER-V4 → policy-result reject
    statement 20: protocol bgp → policy-result accept
info from running routing-policy prefix-set CUSTOMER-V4
    prefix 198.51.100.0/24 mask-length-range exact
```

Group → policy → prefix-set: the customer prefix is rejected toward Lagoon.

Remove the override, then check the service

WSL `make enter NODE=edge01.bob1.reefnet.test`

Router CLI

```
enter candidate

delete / network-instance default protocols bgp group lagoon-v4 export-policy

diff

commit now
```

`lagoon-v4` then inherits `EXPORT-BGP`. Do not edit the policies, leave `lagoon-v6` alone.

Verify from both probes

Leave the router with `quit`.

WSL `make enter NODE=probe01.bob1.lagoontransit.test`

Inside the Lagoon probe

```
curl --fail --max-time 5 -4 http://198.51.100.10/;
curl --fail --max-time 5 -g -6 'http://[2001:db8:100::10]/';
```

Use `exit`, then in WSL:

```
make enter NODE=probe01.bob1.pacifictransit.test;
```

Repeat both HTTP checks. Return with `exit`, then run `make test`.

Important: our lab leaks a route

edge01 advertises to Lagoon Transit (`192.0.2.2`):

Prefix	AS path	Learned from	Gao–Rexford
`198.51.100.0/24`	65000 65010	customer Ocean Research	send to everyone
`203.0.113.128/25`	65000 65200	provider Pacific Transit	**customers only**

During the fault, the leaked route was the **only** route Lagoon received.
Cause: `EXPORT-BGP` accepts every BGP route (`match protocol bgp`).
On a real network, ReefNet would carry transit between its two providers.

Record impact, evidence and recovery

A useful update states:

impact → evidence → action → verification

For example:

> IPv4 through Lagoon Transit was missing because the customer route was filtered on export. The policy attachment was removed and reachability now succeeds from both external probes. IPv6 was unaffected.

RCA: ask why until you can prevent it

RCA = Root Cause Analysis

Why?	ReefNet incident
Lagoon IPv4 unreachable	`198.51.100.0/24` not exported to `192.0.2.2`
Not exported	Group override `BLOCK-CUSTOMER-V4` on `lagoon-v4`
Override present	A configuration change attached it
Root cause: change not caught	No export check per upstream and address family

Write an RFO after the incident

RFO = Reason for Outage

Record	ReefNet incident
Impact	Lagoon IPv4 unreachable
Cause	Group override `BLOCK-CUSTOMER-V4` on `lagoon-v4`
Recovery	Override removed, HTTP verified
Prevention	Test IPv4 and IPv6 exports

FCAPS: five management areas

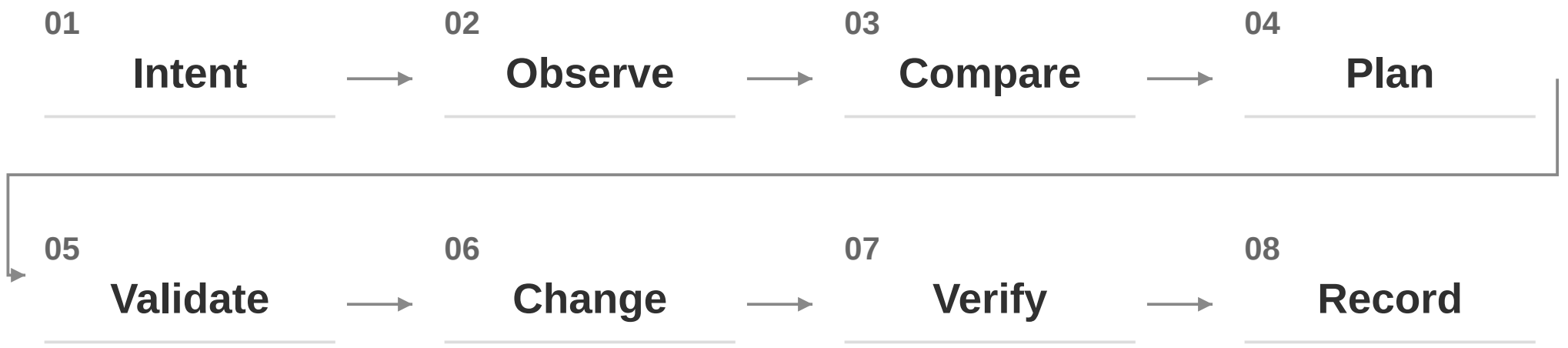
- **Fault**: detect, isolate and recover
- **Configuration**: control intended state
- **Accounting**: attribute resource use
- **Performance**: measure service and capacity
- **Security**: control access and change

Today: a **Fault** (Lagoon IPv4 lost) caused by a **Configuration** change (group override).

Same incident, now through an API

This morning we repaired a router through its CLI.
Now we will use NetBox and gNMI.

The safe change loop

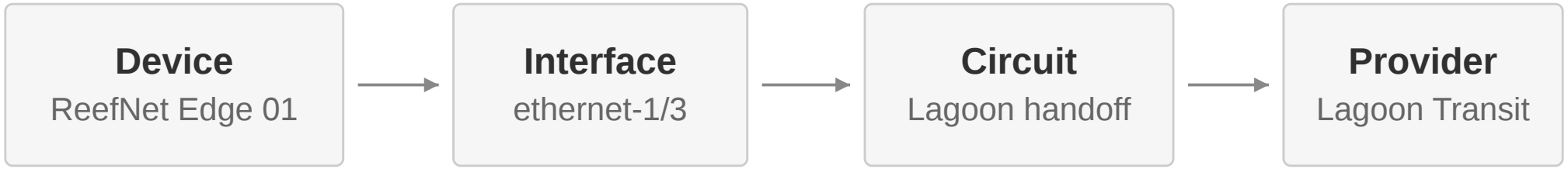


Failed verification: record actual state, back out, verify again.

Step 7 runs this loop: `gnmic get`, `set --dry-run`, `set`, `get`.

Inventory gives the target context

An inventory is a graph of relationships.



Join the intended circuit to the actual change target.

Read the device inventory

WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01

```
AUTH="Authorization: Bearer $(cat /state/netbox-token)";
API=http://netbox.bob1.reefnet.test:8080/api;
curl -fsS -H "$AUTH" "$API/dcim/devices/?name=edge01.bob1.reefnet.test" > device.json;
jq '{count, devices: [.results[] | {name, primary_ip4}]}' device.json;
```

Expect one device with `primary_ip4` `172.20.20.11/24`. Keep the token private.

Read the circuit intent too

WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01

```
AUTH="Authorization: Bearer $(cat /state/netbox-token)";
API=http://netbox.bob1.reefnet.test:8080/api;
curl -fsS -H "$AUTH" "$API/circuits/circuits/?cid=BOB1-LAGOON-001" > circuit.json;
jq '.results[] | {cid, provider: .provider.name, status: .status.label}' circuit.json;
```

NetBox: `BOB1-LAGOON-001` is active.

The router: session to `192.0.2.2` up, `198.51.100.0/24` not exported.

Follow the inventory to the change

Each NetBox query returns a record. We keep one fact and use it for the next query.

`GET /api/…`	**We keep**
`dcim/devices/?name=edge01.bob1.reefnet.test`	Management IP `172.20.20.11` on `mgmt0`
`dcim/interfaces/?device=edge01.bob1.reefnet.test`	`ethernet-1/3` is cabled to Lagoon Transit
`ipam/asns/?asn=65100`	AS 65100 is Lagoon Transit, the BGP peer on that port
`ipam/prefixes/?prefix=198.51.100.0/24`	Ocean Research service prefix

Change input: on `edge01`, peer AS 65100 must receive `198.51.100.0/24`.

Choose the automation mechanism

Layer	Examples	On the BOB1 routers
CLI automation	Netmiko	SSH to the SR Linux CLI
Data models	YANG: native, OpenConfig	`srl_nokia-*` and OpenConfig models
Model-driven protocols	NETCONF, RESTCONF, gNMI	gNMI on port 57400, NETCONF on 830, no RESTCONF

Netmiko: automate the CLI you already know

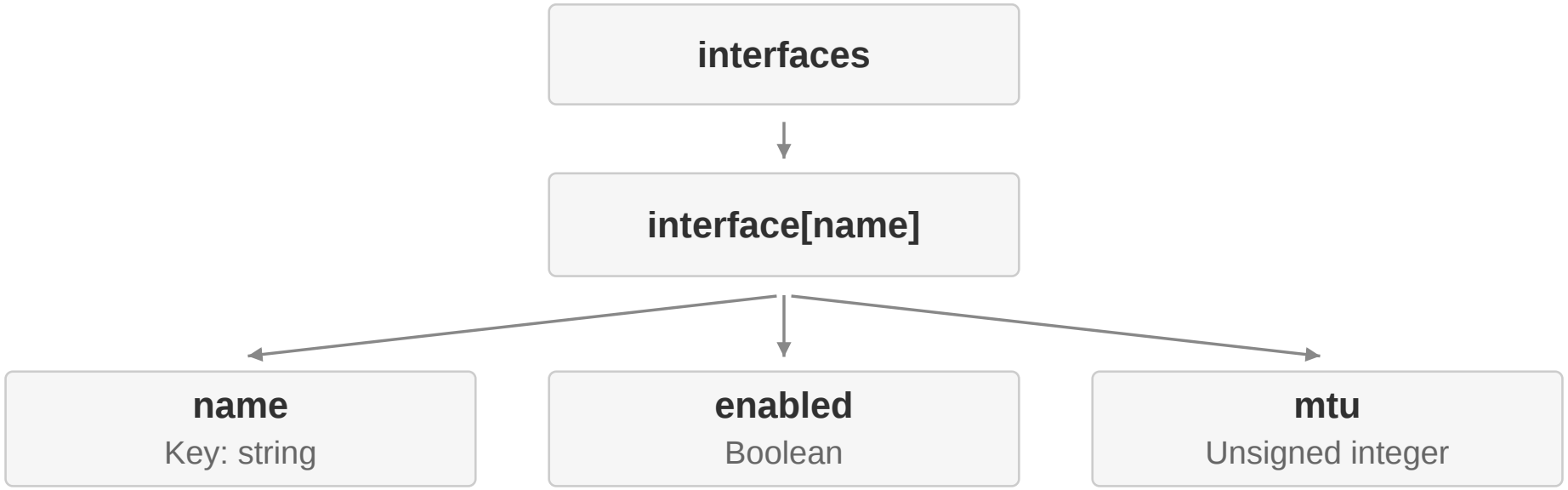
Illustrative SSH read on SR Linux:

```
from netmiko import ConnectHandler

with ConnectHandler(**device) as conn:
    output = conn.send_command(
        "show network-instance default protocols bgp neighbor 192.0.2.2 advertised-routes ipv4")
    assert "198.51.100.0/24" in output
```

Netmiko handles the session. You still parse text.

YANG describes management data



YANG for our BGP group

From `srl_nokia-bgp.yang` on edge01, shortened:

```
list group {
  key "group-name";
  leaf group-name { ... }
  leaf peer-as { ... }
  leaf-list export-policy {
    ordered-by user;
    max-elements 14;
    type leafref { path "/routing-policy/policy/name"; }
  }
}
```

The key makes the path `group[group-name=lagoon-v4]`.
The leafref rejects a policy that does not exist: try `NO-SUCH-POLICY`.

OpenConfig: one model for many vendors

Six OpenConfig models loaded on edge01:

Model	Covers
`openconfig-interfaces`	Admin and oper state, counters
`openconfig-network-instance`	VRFs, BGP, OSPF, static routes
`openconfig-routing-policy`	Prefix sets and policies
`openconfig-platform`	Chassis, line cards, transceivers
`openconfig-lldp`	Neighbor discovery
`openconfig-system`	Hostname, AAA, NTP, logging

Same router, two models

gNMI reads on edge01:

Data	Native SR Linux	OpenConfig
ethernet-1/3 state	…/`oper-state`: `up`	…/`state/oper-status`: `UP`
BGP group lagoon-v4	full configuration	empty

Paths: `/interface[name=ethernet-1/3]/oper-state` and `/interfaces/interface[name=ethernet-1/3]/state/oper-status`.

NETCONF and RESTCONF are siblings

	NETCONF	RESTCONF
Standard	RFC 6241 (2011)	RFC 8040 (2017)
Transport	SSH, port 830	HTTPS
Encoding	XML	JSON or XML
Operations	`get-config`, `edit-config`, `lock`, `commit`	`GET`, `POST`, `PUT`, `PATCH`, `DELETE`
Datastores	running, candidate, startup	one view, no commit step
BOB1 routers	enabled	not offered

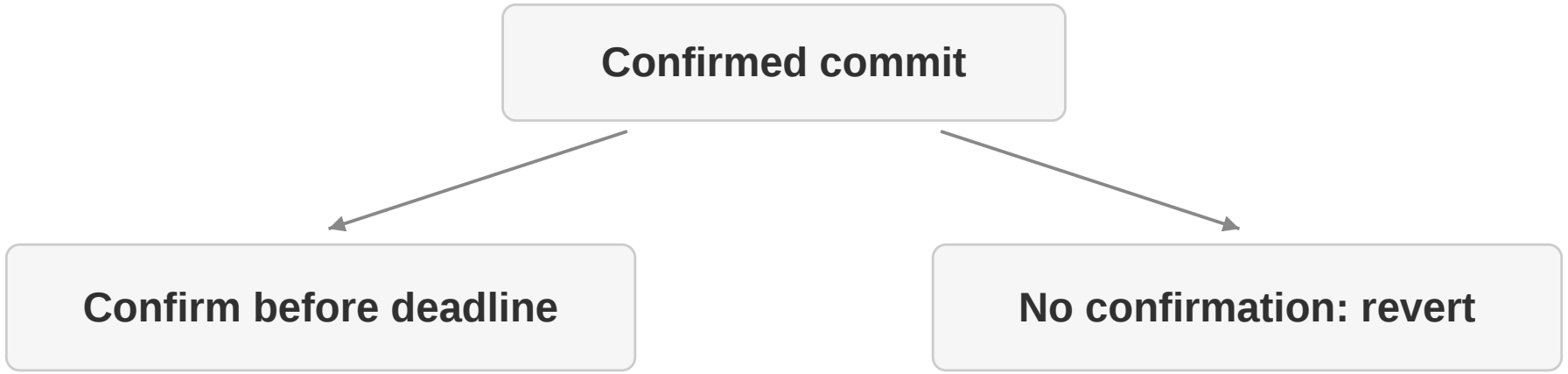
NETCONF edits a datastore

Our repair over SSH port 830, tested on edge01:

```
<edit-config>
  <target><candidate/></target>
  <config>
    <network-instance xmlns="urn:nokia.com:srlinux:net-inst:network-instance">
      <name>default</name>
      <protocols><bgp xmlns="urn:nokia.com:srlinux:bgp:bgp"><group>
        <group-name>lagoon-v4</group-name>
        <export-policy nc:operation="delete">BLOCK-CUSTOMER-V4</export-policy>
      </group></bgp></protocols>
    </network-instance>
  </config>
</edit-config>
```

`lock` → `edit-config` → `commit` → `unlock`, each answered with `<ok/>`. Then verify the service.

Confirmed commit

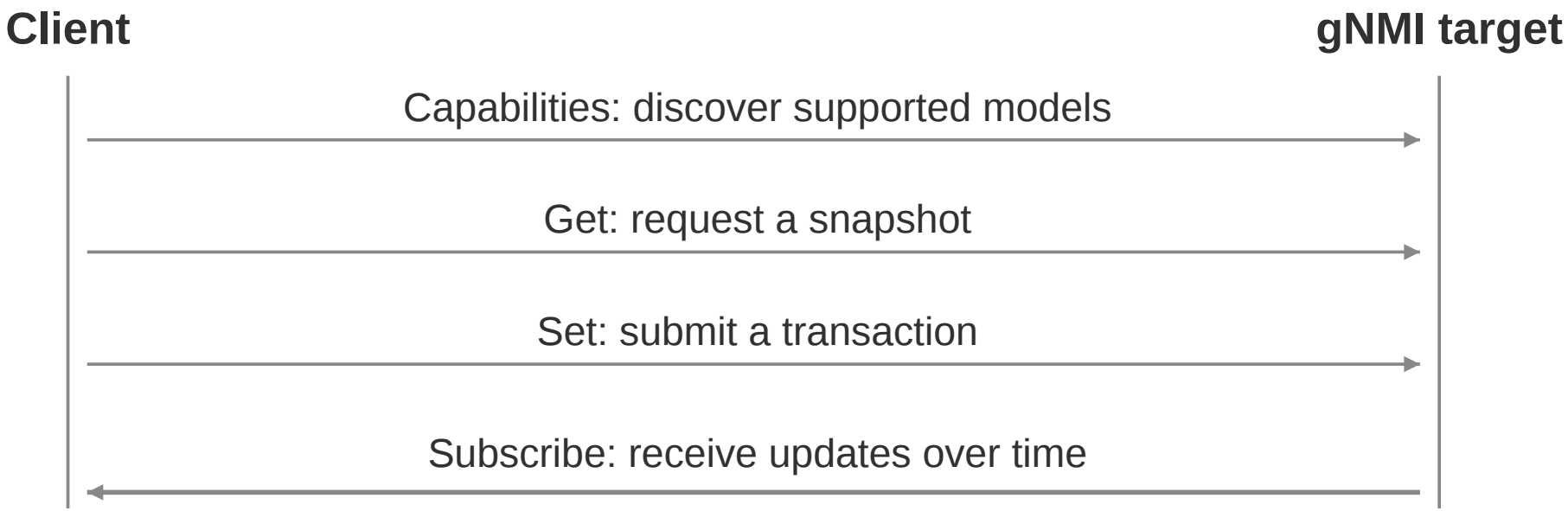


Verify service before confirming. Requires target support.

No confirmation before the timeout: the router reverts. Test that revert.

gNMI defines four RPCs

RPC = remote procedure call



Read a gNMI path

Our repair path on edge01:

```
/network-instance[name=default]/protocols/bgp/group[group-name=lagoon-v4]/export-policy
```

- `[name=default]` selects one network instance.
- `[group-name=lagoon-v4]` selects one BGP group, by its YANG key.
- `export-policy` is the leaf-list we read or delete.
- The **target** `edge01.bob1.reefnet.test:57400` selects the router.

gNMI Set

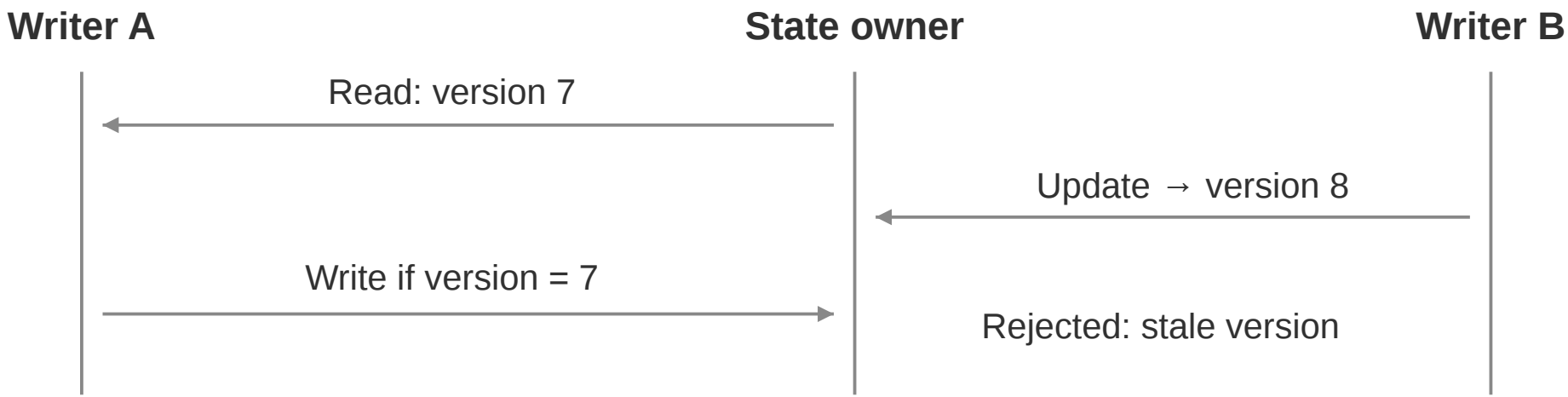
One request, one target, one transaction.

Result	Target behavior
Success	Apply all requested changes
Failure	Restore the pre-request state
Timeout	Outcome unknown to the client

Read back after a timeout. On SR Linux, Set applies immediately: no commit step.

Prevent a lost update

Two operators read `lagoon-v4`, then both write. The later write silently undoes the earlier one.



Check and write must be atomic at the state owner.

Fix: block the second writer, or write only if your read is still current.

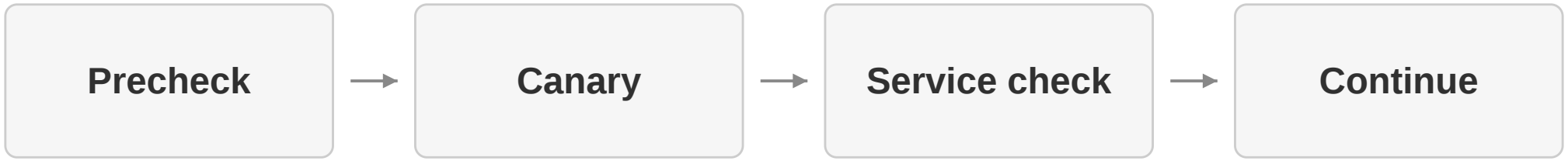
Block the second writer

Interface	Protection	Tested on edge01
CLI	`enter candidate exclusive`	Second CLI session refused
NETCONF	`lock` → read → `edit-config` → `commit` → `unlock`	Second lock: `lock-denied`
gNMI Set	none	Set went through while NETCONF held the lock

Keep one writer per router and read back after every write.

Plan the maintenance window

Method of Procedure (MOP): steps, checks and recovery ownership.



Stop time + recovery owner apply at every gate.

Plan the repair verification

Check	Expected
Lagoon: `198.51.100.0/24`	Export restored
Lagoon: `2001:db8:100::/48`	Still exported
Pacific IPv4 path	Still healthy
Both probes, both families	Service responds

Exercise: repair it through gNMI

Find ReefNet Edge 01 in NetBox. Read its configuration and justify a change.

In WSL:

```
make fault-routing;
make task STEP=6;
```

Lab not running? `make operations` starts it with the fault already in place.

60 minutes: steps 6–8. Repair through gNMI and verify the service from both probes. Use `make hint` for investigation commands.

Exercise overview

Apply the repair through gNMI

WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01, after `get` confirmed `BLOCK-CUSTOMER-V4`

```
export GNMIC_ADDRESS=edge01.bob1.reefnet.test:57400;
export GNMIC_USERNAME=admin GNMIC_PASSWORD='NokiaSrl1!';
export GNMIC_SKIP_VERIFY=true GNMIC_ENCODING=json_ietf GNMIC_TIMEOUT=10s;
GROUP='/network-instance[name=default]/protocols/bgp/group[group-name=lagoon-v4]';
gnmic set --delete "$GROUP/export-policy";
```

Read back the group. Check Lagoon’s route and HTTP from both probes.

Beyond one router: execute and recover

What changes when one change targets many routers?

Bound the rollout

1. Re-read live preconditions.
2. Change one representative canary.
3. Verify the service before expanding.

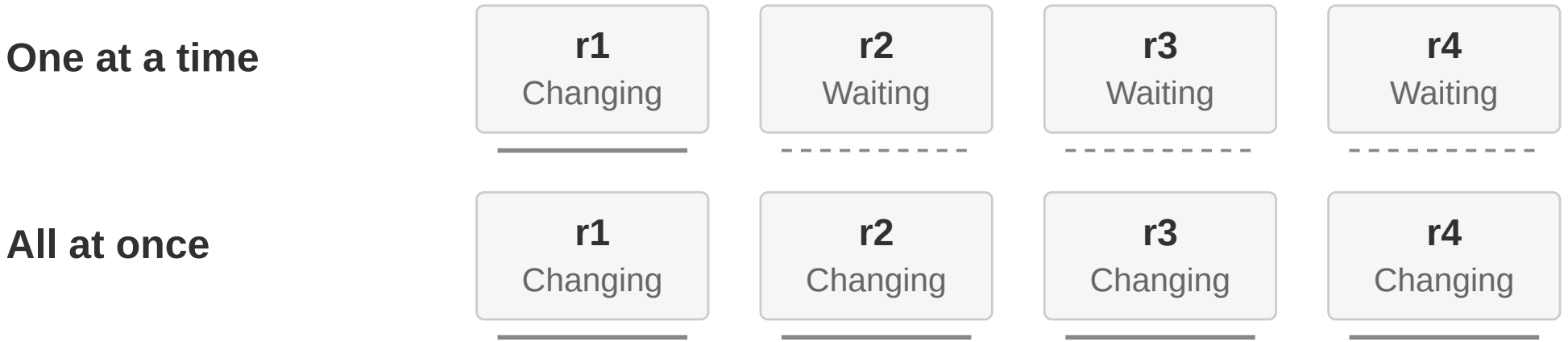
Set a stop time. Keep the backout path reachable.

Give automation a bounded identity

Identity	Read	Write
Collector	Required state	No
Change service	Prechecks + verification	Approved scope
Reviewer	Evidence	No

In the lab, `gnmic` logs in as `admin`, with full write access. A collector should not.

A fleet changes the failure model



How many devices can this change affect before the next check?

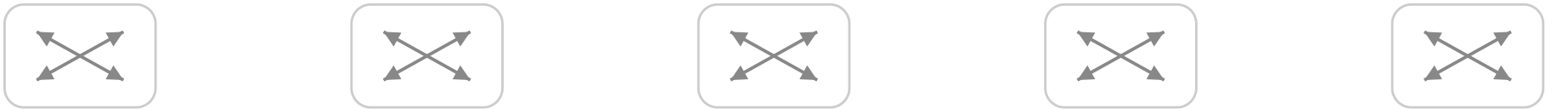
Run it twice. What changes?

Idempotence, for a fixed operation F and state s:

$$F(F(s)) = F(s)$$

Deleting the `lagoon-v4` export-policy twice leaves the same state: no override.

One router timed out after the write



r1	r2	r3	r4	r5
Verified	Verified	Timed out	Waiting	Waiting

Progress is per device. A timeout is not a rollback.

Which device needs a fresh read before any retry?

Which state are we restoring?

“Restore last Tuesday” is not a precise instruction.

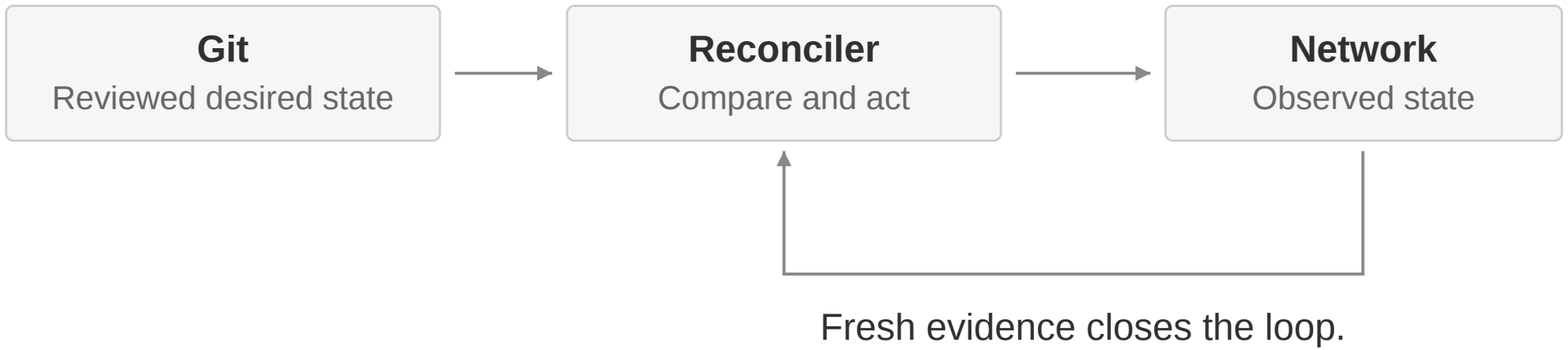
Golden configuration	Backup
Approved baseline	Captured device state

Keep identity, time, version and checksum outside the device.

Keep an audit trail

```
{
  "actor": "course-automation",
  "target": "edge01.bob1.reefnet.test",
  "change_id": "CHG-017",
  "deleted_override": "BLOCK-CUSTOMER-V4",
  "inherited_policy": "EXPORT-BGP",
  "lagoon_ipv4_probe_passed": true
}
```

GitOps gives us a reviewed desired state



Who Broke the Internet?

Everything is redundant until the shared dependency breaks.

Which safeguard would have changed the outcome?

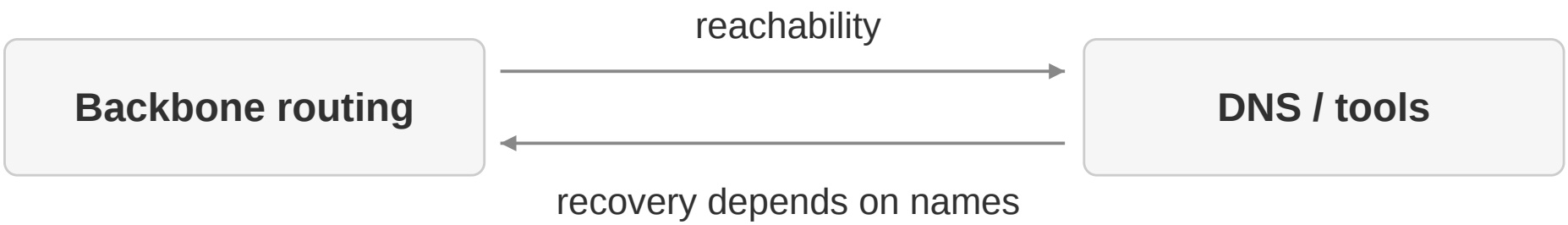
Facebook: the backbone took DNS with it

4 October 2021

1. An audit-tool bug allowed a maintenance command to disconnect the backbone.
2. DNS sites lost data-center reachability and withdrew BGP routes.
3. Running DNS servers became unreachable.

Facebook, Instagram and WhatsApp went offline.

Facebook: recovery lost its own tools



Break the loop before the outage.

Normal and out-of-band access failed. Engineers had to go onsite.

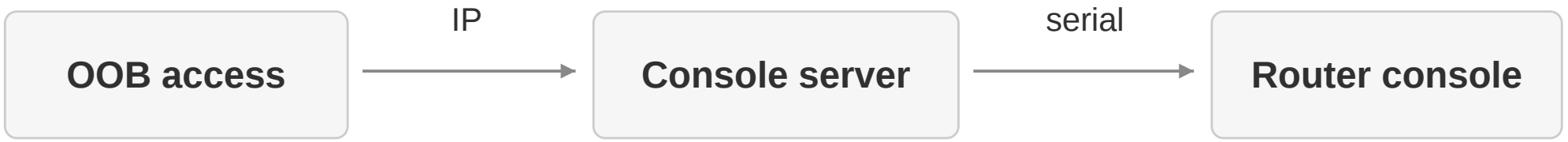
They restored the backbone, then brought services back under controlled load.

Can you still reach the router?

Management path	Uses
In-band	Production network
Out-of-band	Separate device or console access

Check shared dependencies: **power, fiber, DNS and authentication**.

Production recovery example: a console path



Power, emergency authentication and reachable credentials still matter.

Test login and repair while the production path is unavailable.

Task: the console is reachable, login fails



AAA uses the failed backbone. Which fallback must exist?

Recovery only works if it was prepared

Dependency	Prepare
Transit	Circuit and provider contact
Console	Server, port and serial settings
Login	Tested emergency credentials
DNS / secrets	Reachable during the outage
Power	Backup along the recovery path

Test that the emergency account can make the repair.

Who do we call, and when?

An **escalation matrix** fixes contacts and triggers before the incident. Illustrative for ReefNet:

Level	Who	Escalate when
1	NOC on shift	Alert or customer report
2	Network engineer on call	No diagnosis after 15 minutes
3	Vendor support	Suspected software or hardware fault
Provider	Lagoon Transit NOC	Fault on circuit `BOB1-LAGOON-001`
Management	Duty manager	SLA breach likely

Remote hands need an exact job

If remote recovery is blocked, onsite staff become your eyes and hands.

```
Location:   BOB1, rack BOB1-R01
Equipment:  edge01.bob1.reefnet.test, ethernet-1/3
Task:       verify label and photograph LEDs
Scope:      inspect only, no disconnect
Return:     timestamp, photos, label mismatch
```

For vendor escalation, add version, symptoms and logs.

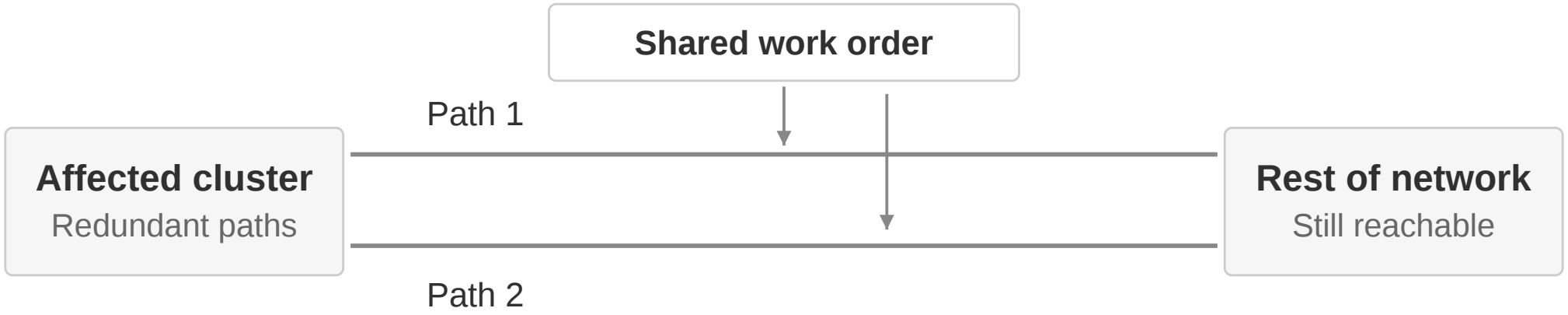
Google Cloud: incompatible optics

1 September 2026, us-central1

New router optics were incompatible with the fabric side.
The work crossed redundant routers before connectivity was checked.

Affected VMs lost connectivity in parts of us-central1-b and us-central1-f.

Google Cloud: one worklist defeated redundancy



All affected paths were unplugged within **13 minutes**.
The worklist omitted router-by-router sequencing and verification.

Recovery: divert traffic, reinstall the original optics, verify links.

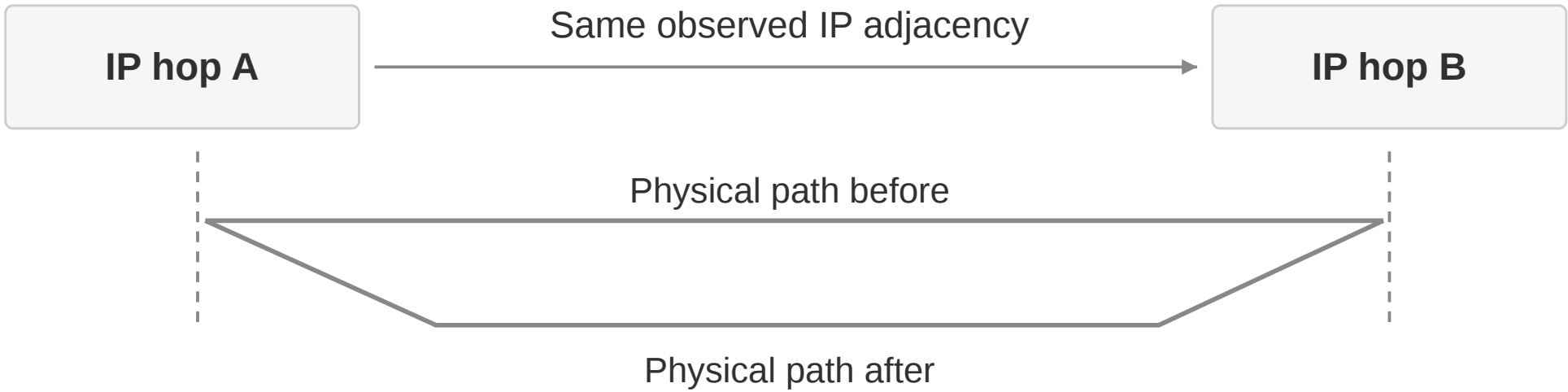
Baltic Sea: paths after the cuts

RIPE Atlas, 17–18 November 2024: Germany–Finland paths after the C-Lion1 cut.

Of 2,141 probe pairs, 21.5% kept identical IP paths.
5.2% of those still got 3 ms or more slower.

No extra packet loss: the networks rerouted, partly below IP.

The IP path can look unchanged



Traceroute alone cannot identify a damaged cable.
Tomorrow you measure RTT from fixed probes, the same method RIPE Atlas uses.

14 October 2026

Monitoring / Observability

In Operations, the customer had to tell us something was wrong.

How could the network have told us first?

What do we want to know?

	Service	Network	Device
Question	Does the exchange work?	Do packets get through?	Is the box healthy?
Examples	Availability, latency	Loss, routing state	Discards, resources
In BOB1	`reefnet_probe_success`	BGP, OSPF, link rates	Interface state, discards

Different signals answer different questions

Signal	Question
Metric	How much or how fast?
Event / log	What changed?
Flow record	Which traffic?
Packet capture	What was on the wire?
Active probe	Does this exchange work?

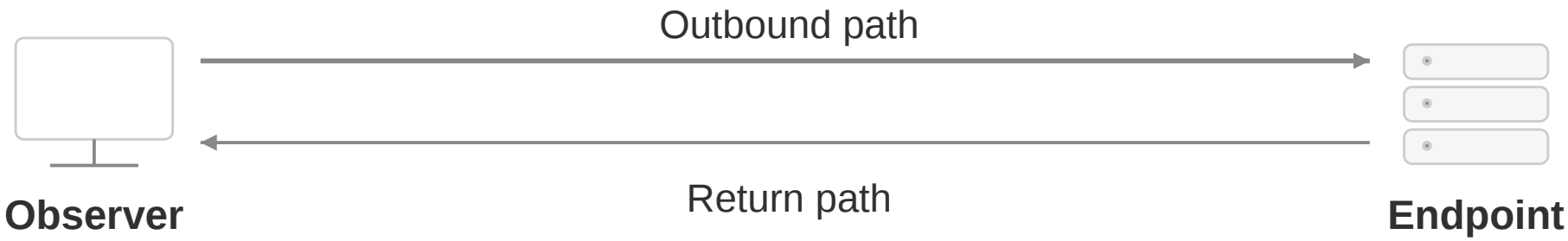
The monitoring brief

What should the BOB1 probes guarantee?

- Detect a Lagoon IPv4 interruption of **5 seconds or longer**.
- Stay within **20 HTTP requests/s** toward the customer.
- Mark stale data **unknown within 10 seconds**.

We check these three requirements against the lab today.

Delay and loss need endpoints



RTT contains both paths and endpoint processing.

Choose the probe from the question

Probe	Tells us
ICMP echo	IP reachability to a responding endpoint
TCP connect	transport handshake
HTTP request	application transaction
traceroute	responding hops along a path

Define the probe before you run it

One BOB1 probe is:

```
source:          probe01.bob1.lagoontransit.test
target:          198.51.100.10
address family:  IPv4
protocol:        ICMP echo
reply wait:      1 s
success:         ping exits successfully
```

Budget the ReefNet probe round

For the smaller BOB1 probe set:

2 sources × 2 families × 3 probe types = 12 checks

All checks of a probe run in parallel. Each check gives up after 1 s.
Then the exporter pauses 1 s: about one round per second, about **4 HTTP requests/s**.
A failing round takes about 2 s. That stays well within the 10 s UNKNOWN threshold.

Video: SNMP basics

Understanding SNMP Agents & Managers, Teltonika Networks (YouTube)

SNMP: manager and agent

	Manager → agent	**Agent → manager**
Messages	Get, GetNext, GetBulk, Set	Trap, Inform
Port	UDP 161	UDP 162
Used for	Polling counters and state	Reporting events

v2c: community string in clear text. **v3:** users, authentication, encryption.

MIB and OID: a number for every value

`1.3.6.1.2.1` = iso.org.dod.internet.mgmt.mib-2

IF-MIB object	OID	Value
`ifName`	`1.3.6.1.2.1.31.1.1.1.1`	`ethernet-1/3`
`ifOperStatus`	`1.3.6.1.2.1.2.2.1.8`	`up(1), down(2)`
`ifInOctets`	`1.3.6.1.2.1.2.2.1.10`	Counter32: wraps after 3.4 s at 10 Gbit/s
`ifHCInOctets`	`1.3.6.1.2.1.31.1.1.1.6`	Counter64

The last number selects the row: `ifHCInOctets.3` is the interface with `ifIndex` 3.

SNMP and gNMI side by side

	SNMP	gNMI
Data model	MIB, numeric OIDs	YANG, named paths
Interface key	`ifIndex`, can change after reload	Interface name
Delivery	Manager polls	Device streams
Events	Trap, Inform	ON_CHANGE subscription
BOB1	Not provisioned	gNMIc, every 1 s

Most production networks still poll with SNMP.

Task: present one network management system

Small groups, one system each:

OpenNMS · Zabbix · LibreNMS · Icinga · Checkmk · PRTG · Nagios Core

20 minutes research, then **3 minutes** per group.

Syslog: what changed?

Observation	Time, host, severity, app and message
Limit	Missing logs do not prove health
ReefNet	SR Linux → Alloy → Loki → Grafana

From `make fault-link`, in Devices & Interfaces:

```
OSPF instance default: Interface ethernet-1/4.0, moved to state DOWN due to event IF_DOWN
```

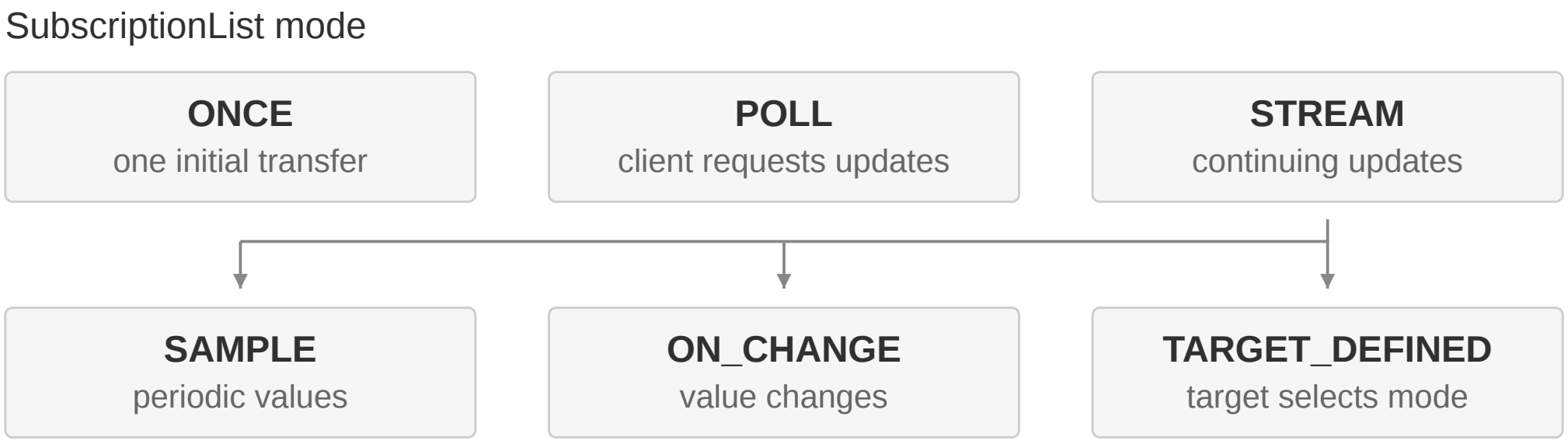
IPFIX: which traffic crossed here?

Observation	Flow keys, counts and time interval
Limit	Typical records omit payload. Sampling affects counts
ReefNet	Not provisioned in BOB1

Streaming state: what changed?

Observation	Target, keyed path, timestamp, update/deletion
Limit	Reconnect cannot restore every missed event
ReefNet	gNMIc SAMPLE: interfaces every 1 s, BGP and OSPF every 2 s

Choose the subscription semantics



Watch counters arrive through gNMI

WSL `make enter NODE=operations01.bob1.reefnet.test`

Inside operations01

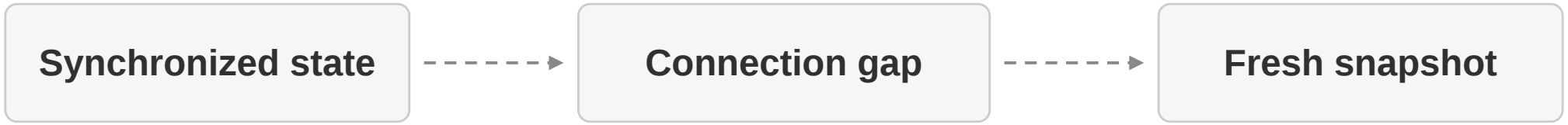
```
export GNMIC_ADDRESS=edge01.bob1.reefnet.test:57400;
export GNMIC_USERNAME=admin GNMIC_PASSWORD='NokiaSrl1!';
export GNMIC_SKIP_VERIFY=true GNMIC_ENCODING=json_ietf GNMIC_TIMEOUT=10s;
IF='/interface[name=ethernet-1/3]/statistics';
gnmic subscribe --path "$IF" --stream-mode sample --sample-interval 5s;
```

Ctrl+C stops the subscription.

Zero, missing or deleted

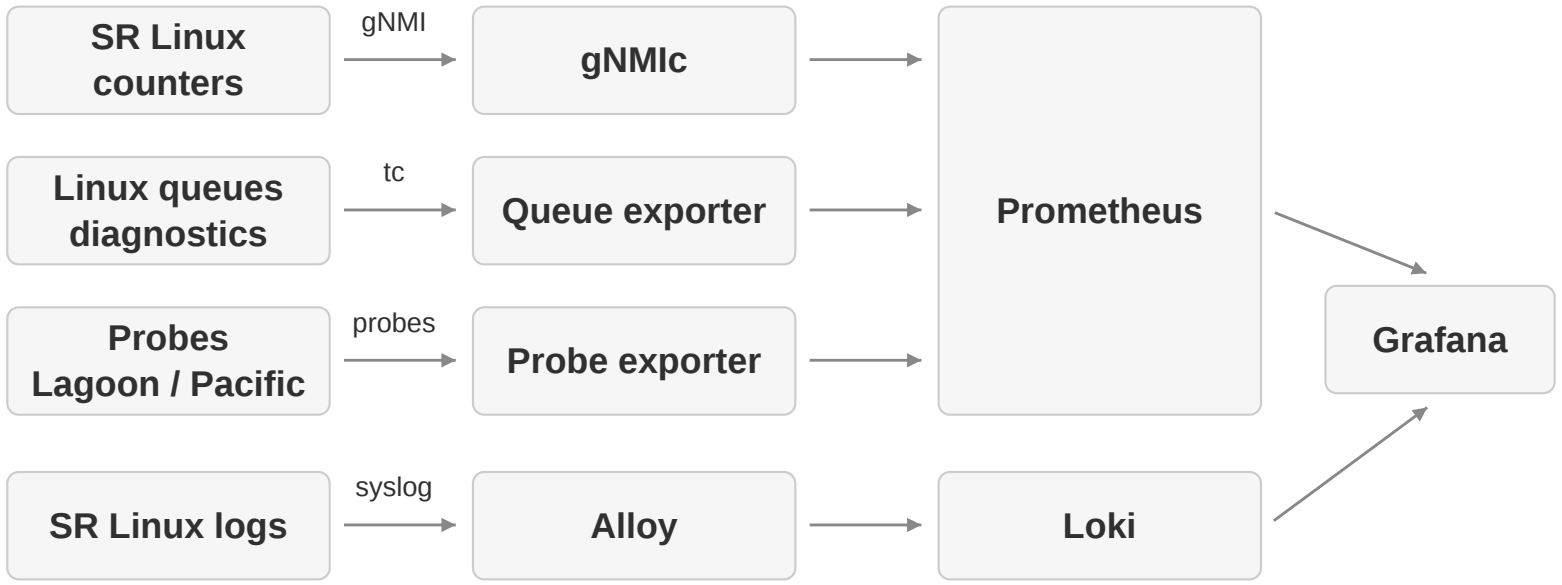
0	—	Deleted
Measured zero	**No fresh sample**	**Object removed**
Valid observation	State unknown	Update the model

Reconnect cannot restore lost history

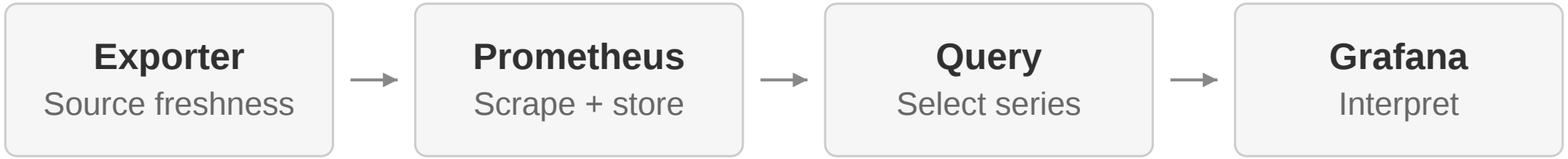


Events during the gap may be irretrievable.

Metrics and logs in ReefNet



Prometheus stores time series



A successful scrape can still return stale source data.

Try PromQL at `http://localhost:9090`.

Cardinality: count ReefNet's series

2	×	2	×	3
sources		IP families		probe types
Lagoon / Pacific		IPv4 / IPv6		ICMP / DNS / HTTP

12 series for one metric

One target: data.oceanresearch.test

`count(reefnet_probe_success)` → **12**

Success, duration and timestamp together → **36 series**.

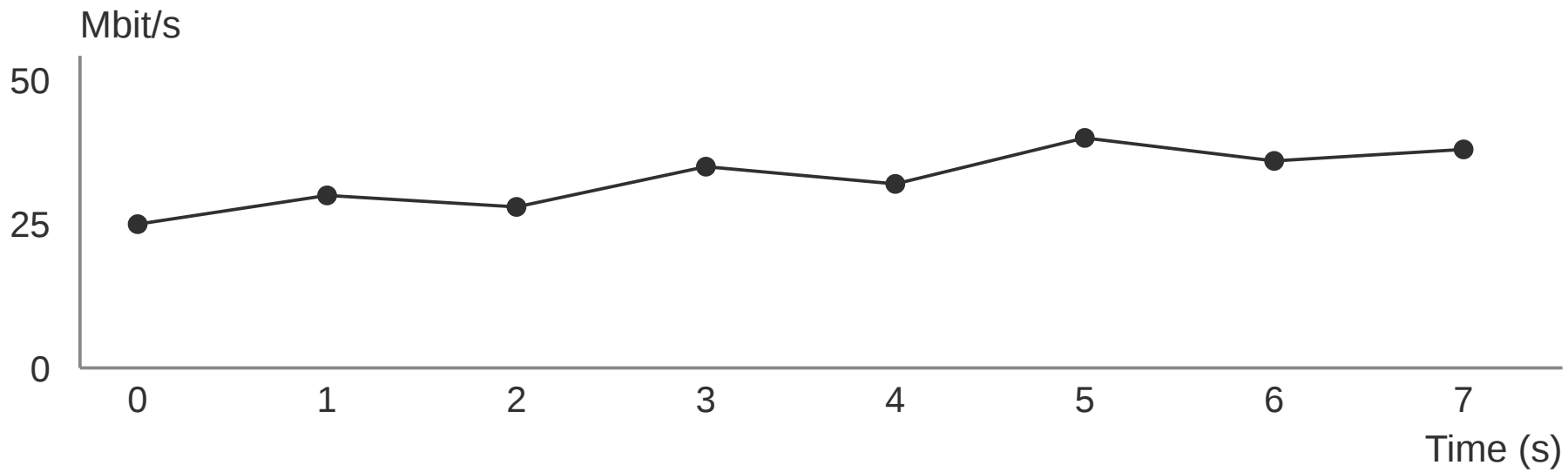
Would you add a request ID?

A proposed change to ReefNet's probe metric:

```
source="probe01.bob1.lagoontransit.test"
probe="icmp"
af="ipv4"
target="data.oceanresearch.test"
request_id="a-new-value-for-every-run"
```

What does `request_id` do to cardinality? Where should it go?

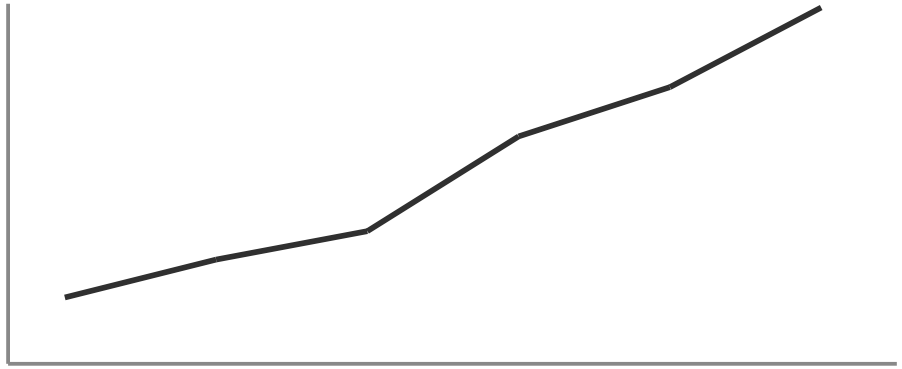
One ReefNet interface, one series




```
reefnet_interface_in_octets_total{source="edge01.bob1.reefnet.test",
interface_name="ethernet-1/3"}
```

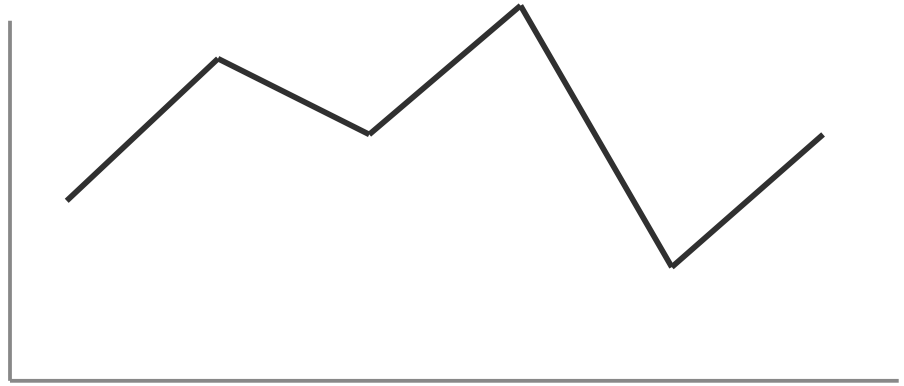
Counter or gauge?

Counter



Bytes on edge01 / ethernet-1/3

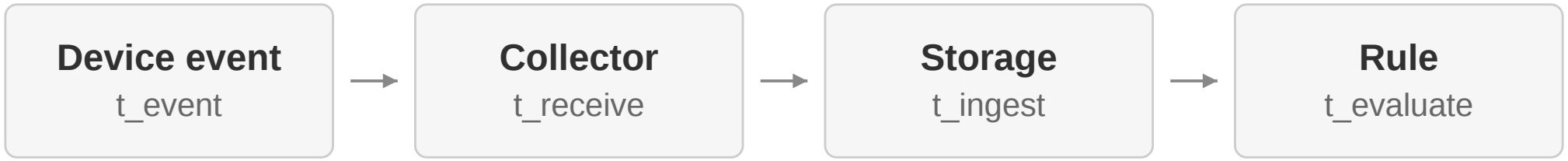
Gauge



DNS-check duration from Lagoon

Timestamps in the pipeline

One event, several clocks.



Clock offset and transport delay are different quantities.

Before subtracting counters

Check identity, timestamps and discontinuities.
A negative delta may mean **reset, wrap or bad data**. Prometheus `rate()` treats any decrease as a reset.
Invalid evidence becomes **unknown**, never a fabricated rate.

Turn ReefNet byte counters into bit/s

Use `in-octets` on `edge01.bob1.reefnet.test`, interface `ethernet-1/3`.

$$rate = \frac{8(C_2 - C_1)}{t_2 - t_1}$$

BOB1: `reefnet_interface_in_bps = rate(reefnet_interface_in_octets_total[10s]) * 8`

Calculate the Lagoon ingress rate

Example `in-octets` on `edge01.bob1.reefnet.test`, interface `ethernet-1/3`, at the 25 Mbit/s baseline.

Time	Byte counter	Reset?
10:00:00	1,000,000	Initial sample
10:00:10	32,250,000	No
10:00:20	20,000	Yes
10:00:32	37,520,000	No

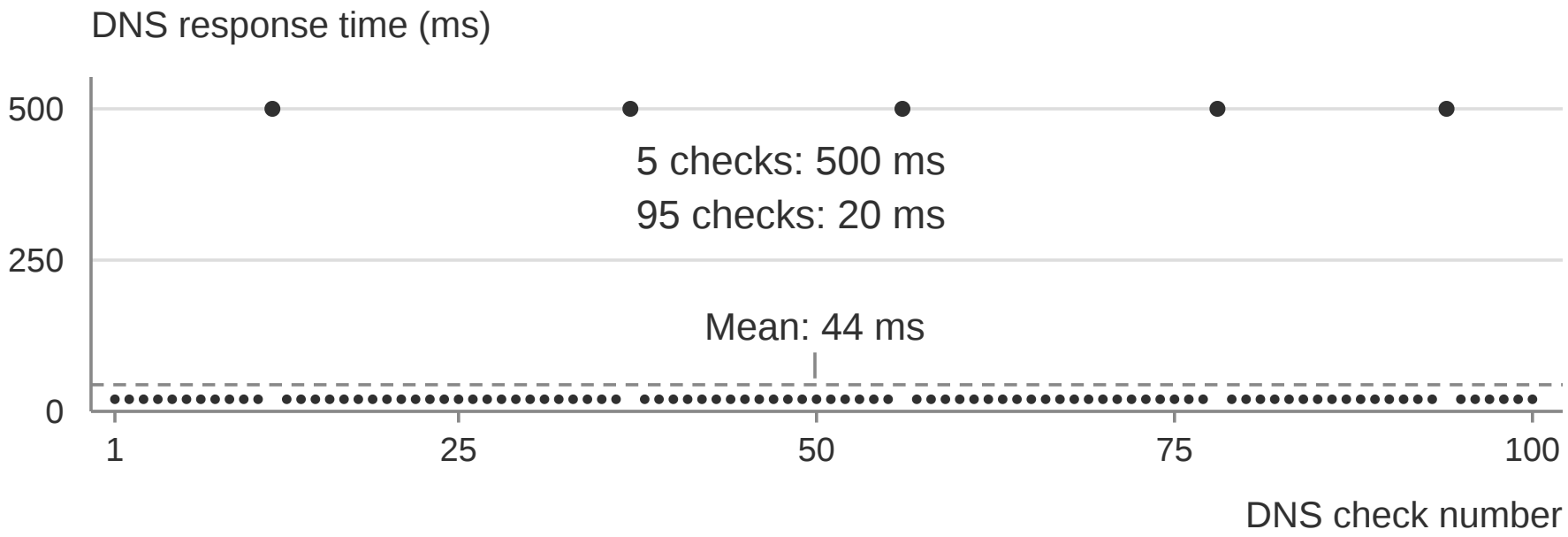
Four minutes: calculate valid rates. Compare them with the 50 Mbit/s handoff.

25 Mbit/s on a 50 Mbit/s handoff

Interval	Rate
00–10 s	8 × 31,250,000 / 10 = **25 Mbit/s**
10–20 s	Reset: **exclude**
20–32 s	8 × 37,500,000 / 12 = **25 Mbit/s**

Both valid intervals use **50%** of handoff capacity.

What Lagoon sees when it probes DNS



Would the **44 ms mean** explain the five long waits?

Read the distribution

View	What it tells us
Median	Middle observation
p95	Threshold covering at least 95%
Histogram	Counts within ranges
Cumulative distribution	Fraction at or below each value

p95 in BOB1, without histograms:

```
quantile_over_time(0.95, reefnet_probe_duration_seconds{probe="dns"}[5m])
```

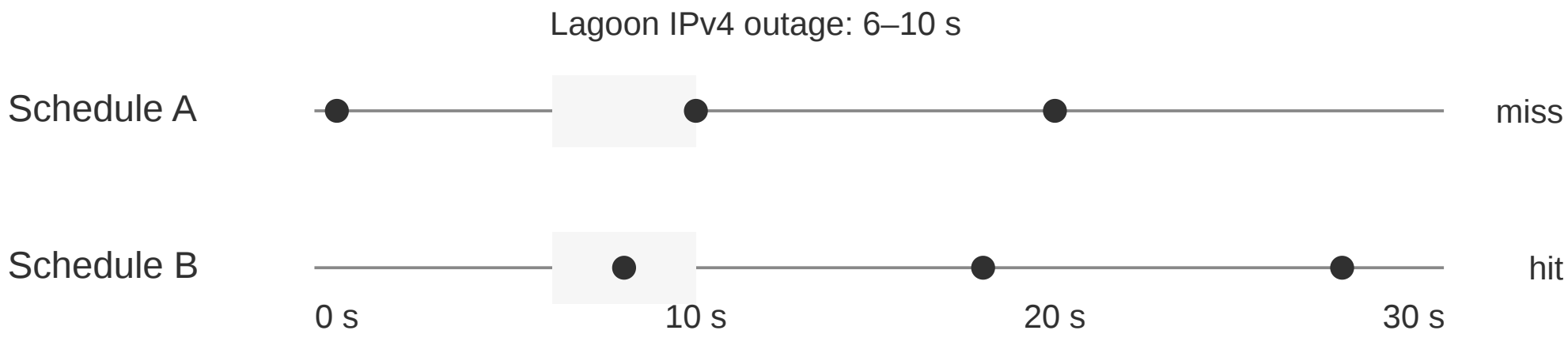
Percentiles do not average

A separate DNS example for `data.oceanresearch.test`. Nearest-rank p95:

Source	Checks	Duration of each check
Lagoon	100	20 ms
Pacific	1	1,000 ms

Average of the two p95s: **510 ms**.
Combined observations: **p95 = 20 ms**.

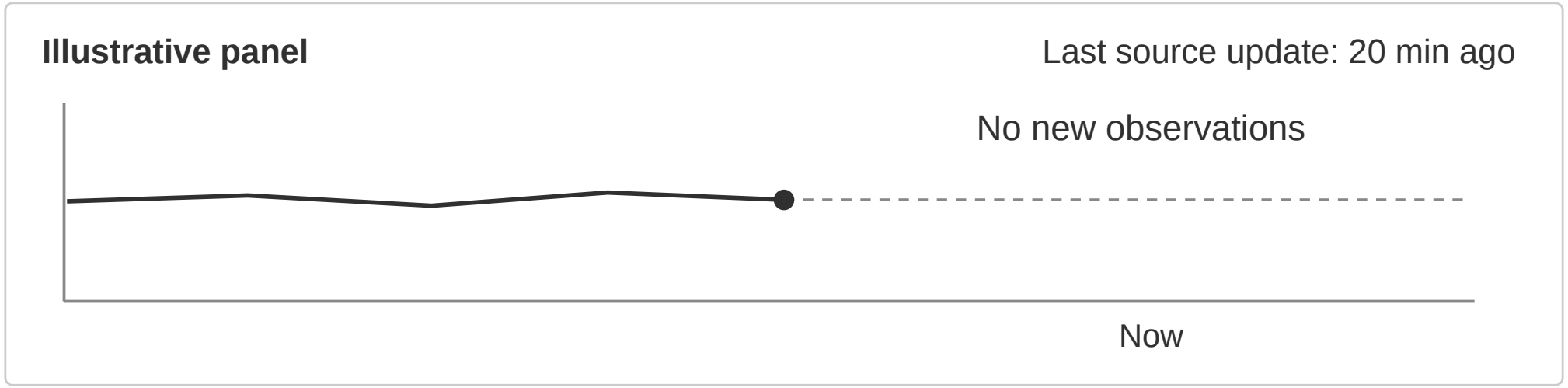
Four seconds can disappear



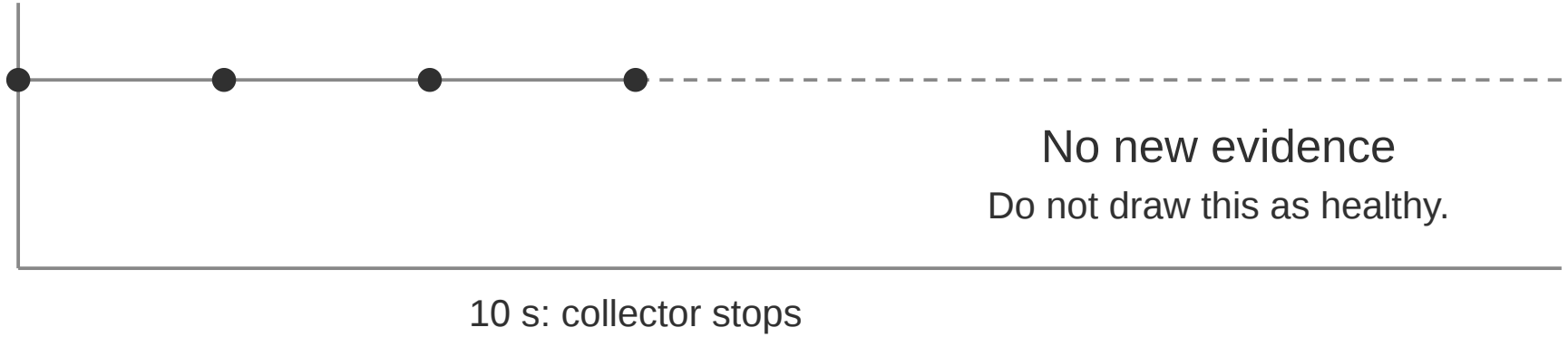
Same 10 s interval. Different onset phase.

With a 10 s interval, a 4 s outage can fall between samples. BOB1 probes about every second and would see it.

Watch the sample age



Missing data on a dashboard



Monitor the monitoring

Component	BOB1 check
Scrape targets	8 targets: gNMIc, 2 probe and 5 queue exporters, all `up` = 1
Probe freshness	Telemetry Health → Oldest probe sample
Router sources	Telemetry Health → Router sources / 5 expected
Queue collection	Telemetry Health → Queue collection

```
up{job=~"gnmic|reefnet-service-probes|reefnet-qdisc"}
```

Choose a dashboard

Question	BOB1 dashboard
Network status and topology	Network Overview
Reachability and response time	Service Health
Sessions, routes and updates	BGP
Core adjacencies and SPF activity	OSPF
Port state, discards and device events	Devices & Interfaces
Rates and utilization	Traffic & Capacity
Collection and freshness	Telemetry Health

What is Grafana telling us?

At **25 Mbit/s IPv4 traffic**, open `http://localhost:3000`.
Choose **ReefNet / BOB1 → Network Overview**.

- What is measured, and where?
- Which units and time window?
- How fresh is it?

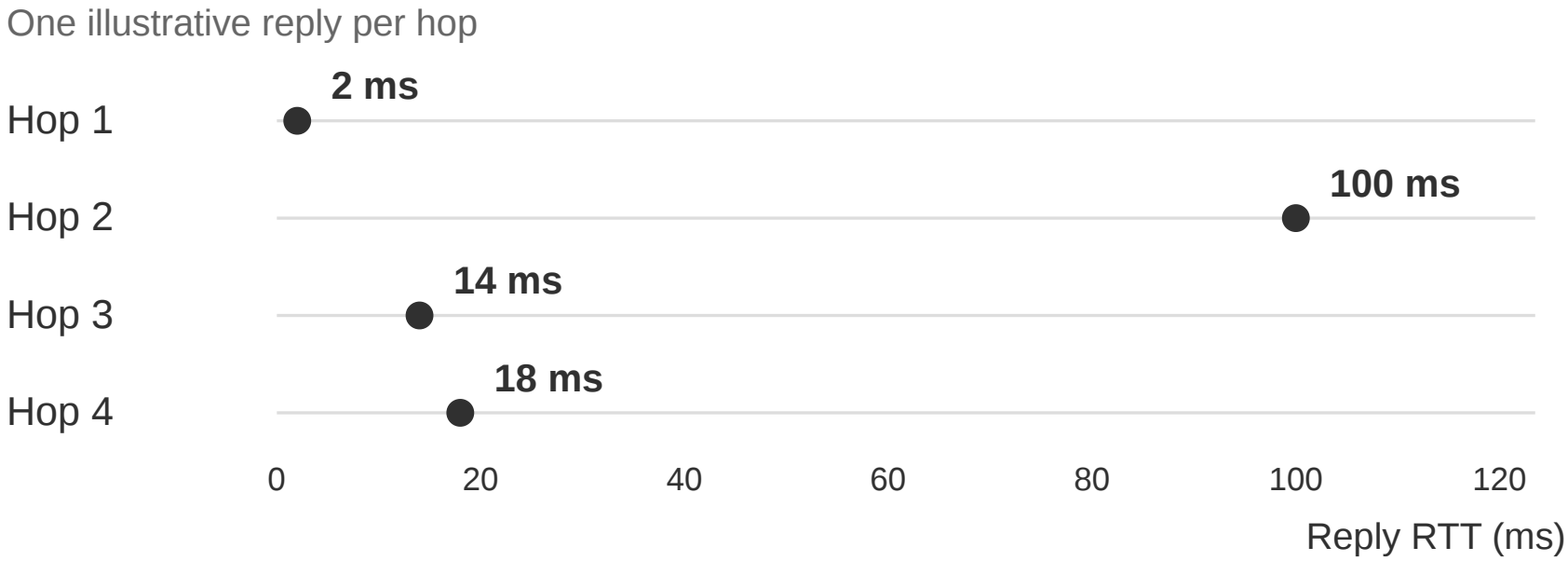
Predict what changes when we send more traffic.

Same symptom, different causes

Lagoon IPv4 fails in both cases.

Evidence	Export rejected	Session down
BGP session to `192.0.2.2`	Established	Not established
`198.51.100.0/24` in edge01's route table	Yes	Yes
Lagoon IPv6	Works	Depends on the IPv6 session

Traceroute: read replies carefully



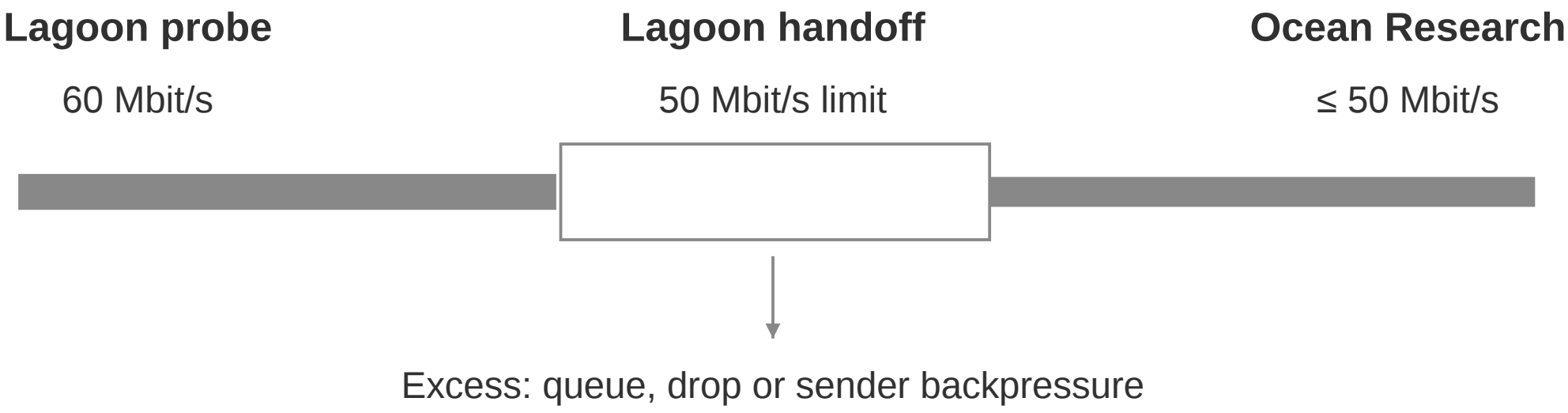
Admin state and oper state

`make fault-link` disables `ethernet-1/4` on ReefNet Edge 01:

```
/interface[name=ethernet-1/4]/admin-state   "disable"
/interface[name=ethernet-1/4]/oper-state    "down"
```

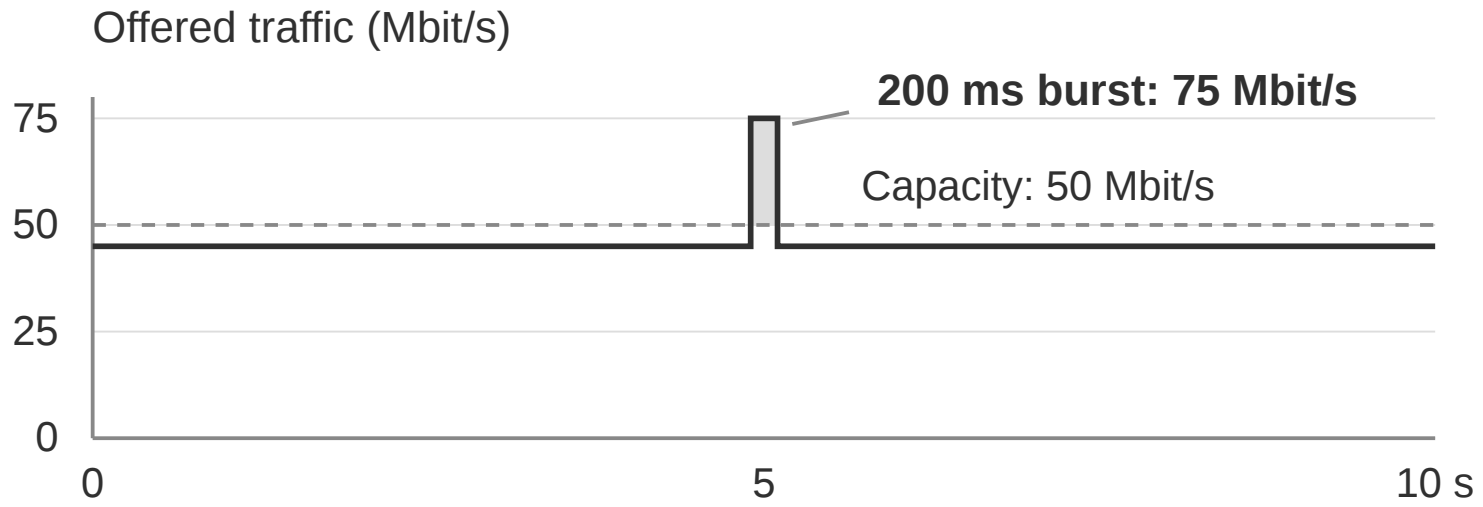
Admin: configured intent. **Oper:** observed behavior.

60 Mbit/s offered, 50 Mbit/s delivered



The customer handoff has 100 Mbit/s, Lagoon only 50 Mbit/s. At 60 Mbit/s the receiver loses about one sixth.

A short burst inside a long average



Dashboard

≈90%

mean output
over 10 s

Synthetic Lagoon example. The brief peak is lost in the average.

Check discards, not only rates.

Where do the packets go?

What happens when offered traffic exceeds the **50 Mbit/s** handoff?

In WSL:

```
cd ~/network-management-and-monitoring && git pull;
make monitoring;
make task;
```

75 minutes: compare load, received traffic and new discards.
Then try link and routing faults and a short burst.

Exercise overview

How much traffic reached the customer?

WSL `make enter NODE=edge01.bob1.lagoontransit.test`

Router CLI: physical interface counters

```
info from state interface ethernet-1/1 statistics
```

WSL after quit `make enter NODE=service01.bob1.oceanresearch.test`

Inside the receiver: Mbit/s and loss

```
tail -n 15 /tmp/iperf-server.log;
```

Compare new egress discards with receiver loss.
Then in WSL: `make traffic-10mbit;`

Keep the simulator out of the result

75 Mbit/s, 1,200-byte UDP datagrams:

≈ **7,800 packets/s**, below the 10,000 packets/s that `traffic.sh` allows

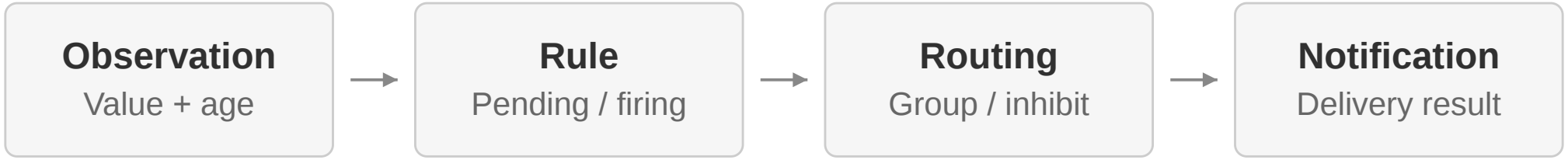
The **50 Mbit/s handoff** should limit first.

Several signals, one incident



One controlled link fault, several observations.

Rule, routing, notification



BOB1 has no Alertmanager: a rule can fire, but nobody is paged.

An alert rule from the lab metric

A rule on the real BOB1 metric, checked with `promtool`:

```
- alert: LagoonIPv4ProbeFailed
  expr: |
    reefnet_probe_success{
      source="probe01.bob1.lagoontransit.test",
      probe="icmp", af="ipv4"
    } == 0
  for: 30s
```

This sustained-failure rule does **not** meet our 5-second detection brief.

Alert on stale observations

```
time() - reefnet_probe_last_run_timestamp_seconds > 10
```

This detects old exported results, matching the 10 s UNKNOWN rule in Grafana.
Missing results need their own rules:

```
count(reefnet_probe_success) < 12
up{job="reefnet-service-probes"} == 0
```

SLI and SLO

Service Level Indicator (SLI): measured service behavior

Service Level Objective (SLO): target over a defined window

$$SLI = \frac{successful\ requests}{eligible\ requests}$$

BOB1, one hour:

```
avg_over_time(reefnet_probe_success{source="probe01.bob1.lagoontransit.test",probe="icmp"
,af="ipv4"}[1h])
```

SLA: the SLO in a contract

Service Level Agreement (SLA): promise to a customer, with consequences.
The internal SLO stays stricter.

	Ocean Research, illustrative
Service	`data.oceanresearch.test` over IPv4 and IPv6
Measured by	ReefNet probes from Lagoon and Pacific
Availability	99.9% per month: about 43 minutes of downtime
Exclusions	Announced maintenance windows
Breach	Service credit

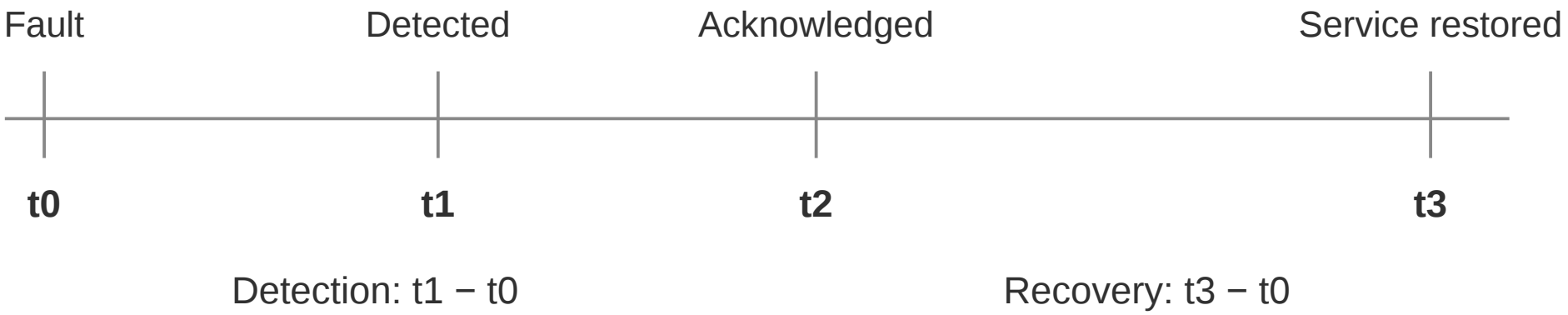
An error budget for the Lagoon probe

IPv4 ICMP from Lagoon to Ocean Research:
99.9% SLO across 10,000 completed checks.

$$10000 \times (1 - 0.999) = 10 \text{ allowed failures}$$

Six failures leave **four**. Report missing checks separately.

Detection and recovery are different clocks



Rehearse a response

Five-minute tabletop during a change:

Vantage point	Service result
Lagoon, IPv4	Fails
Pacific, IPv4	Works
Lagoon, IPv6	Works

Name the first observation, incident owner and backout trigger.

Test the monitoring brief

Requirement	Test
Detect 5 s interruptions	Vary onset phase
≤ 20 requests/s	Count requests and retries
Unknown within 10 s	Stop a probe exporter, watch Service Health
Useful diagnosis	Compare route loss and drops

What would you tell the customer?

Was it congestion, a missing route or missing data?
Choose the measurements that explain your answer.

Note your observations, then stop the lab:

```
make traffic-stop;
make down;
```

Use `make monitoring` to start a fresh experiment on a configured lab machine.

30 October 2026

Research questions and sources

BOB1 runs its 12 probe checks about every second.
With 200 targets and the same 20 requests/s, each target gets a check every 10 s.

A 5-second outage can then fall between two checks. Which schedule finds more?

Leave with a testable research brief

1. State a question that a comparison can answer.
2. Find prior work and evidence that fit the question.
3. Choose a baseline and expose the biggest feasibility risk.

Our running example: **short outages missed between probes**.

Some project ideas

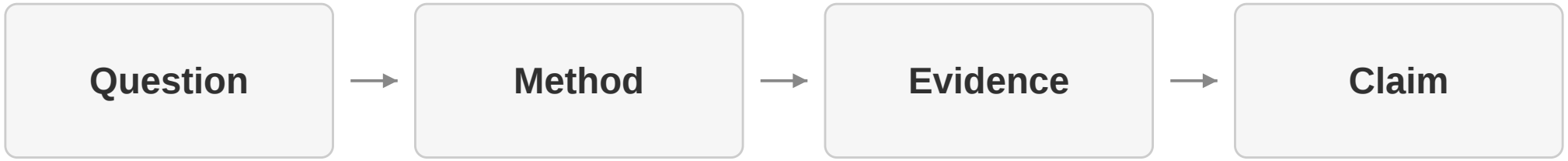
Area	Possible comparison
Monitoring	Polling vs streaming for short failures
Digital twins	Emulation vs modeling before deployment
Automation	Automated vs manual emergency repair
AI for networking	LLM troubleshooting vs a runbook

From a problem to a research question

At equal cost, which schedule catches short outages better?

Compare	Periodic vs randomized
Keep equal	Probe budget, workload, endpoint
Measure	Detection fraction and delay
Bound the claim	Tested fault durations and conditions

What makes a scientific argument?



The claim cannot be stronger than its weakest evidence boundary.

What kind of paper will you write?

Approach	Contribution
Measurement	Describe behavior
Comparison	Evaluate methods fairly
Reproduction + extension	Check and extend a result
System study	Build or investigate a system

Task: challenge the claim

> "Randomized polling detects outages faster."

Two minutes:

1. Name a result that would contradict this.
2. Specify outage durations, baseline and measured outcome.
3. Choose a success criterion that allows the method to lose.

Plan the measurement from the claim

Claim	Metric	Evidence
faster detection	detection delay	repeated known faults
lower load	bytes/s or requests/s	collector + target
fewer misses	detection rate	known injected events

Observation or intervention?

Observational study	Controlled experiment
polling gaps coincide with missed events	deliberately vary the polling schedule
shows association	tests an intervention

A **confounder** changes both condition and outcome.

Find the closest comparison

Search by **problem, mechanism and method**. Follow citations.

Compare assumptions, baselines and evidence.
Record your queries and selection criteria.

“Nobody has done this before” is not a literature review.

Research venues close to this course

Venue	Typical contribution
IEEE/IFIP NOMS	operating, managing and automating networks
CNSM	management systems, telemetry and orchestration
ACM IMC	measuring and understanding Internet behavior

Who develops Internet technology?

IETF

Working Groups engineer interoperable Internet protocols and standards.

IRTF

Research Groups study long-term questions about Internet technology.

Where do operators compare notes?

RIPE Meetings
Routing, addressing, measurement and Internet coordination

DENOG and NANOG
Backbone operations, routing, peering and incidents

NAF and AutoCon
Network automation, orchestration and operational tooling

Read in three passes

Pass	Look for
Relevance	Question and main result
Method	Signal, baseline and experimental unit
Challenge	Missing evidence and transfer limits

Read the claim before the graph

Trinocular probes **IPv4 /24 blocks**, not applications.

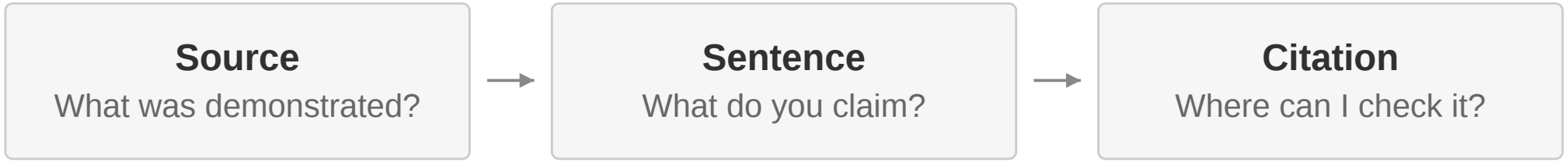
Check	Ask
Scope	Which blocks and vantage points?
Method	Which outage definition and budget?
Validation	Which independent evidence?

Match the source to the claim

Claim	Evidence source
Required protocol behavior	Specification and exact section
Implemented feature	Versioned code or official documentation
Measured improvement	Paper and evaluation data
Production failure	Attributed operator report

For RFCs: check status, updates, errata and normative language.

Does the citation support the sentence?



Paraphrase the idea, not the word order

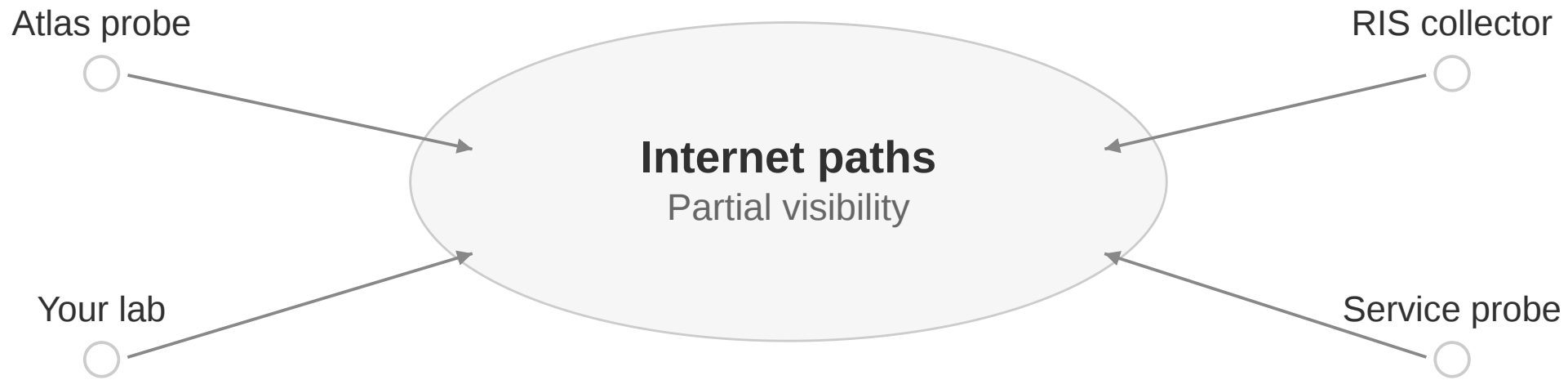
Approach	Assessment
Replace a few words	Still follows the original
Explain from your notes	Paraphrase, with attribution
Copy exact wording	Quote, with location

A citation alone does not mark a quotation.

Make evidence traceable

Artifact	Record
Software	Version or commit and repository
Dataset	Release, identifier, access conditions
Documentation	Exact page, version and section
Figure	Data and units, or source and adaptation

Public data gives us another vantage point



RIPE Atlas: active probes. **RIS:** public BGP observations.

Read one public routing record

NETHINKS (AS8319), Fulda. In a shell with curl and jq, such as operations01:

```
URL=https://stat.ripe.net/data/routing-history/data.json;
Q='resource=AS8319&starttime=2025-09-05T00:00:00&endtime=2025-09-07T00:00:00';
curl -fsS -o as8319.json "$URL?$Q&include_first_hop=true&min_peers=10";
jq '.data.by_origin[0].origin' as8319.json;
jq '.data.by_origin[0].prefixes[0] | {prefix, interval: .timelines[0]}' as8319.json;
```

What did the collector actually see?

AS8319 example, checked 4 October 2026:

```
origin + first hop: 6939 8319
prefix:             149.218.0.0/17
interval, UTC:      5 Sep 00:00 to 7 Sep 07:59:59, 2025
full_peers_seeing: 11
```

Route visibility is not service availability.

Make the public measurement reproducible

Keep the details another person needs to repeat it:

- UTC time window
- unaffected comparison
- query or measurement IDs
- filters and thresholds

Save them with the conclusion.

Plan responsible measurement

Prefer public data or your own lab.

For active probes, agree on **target, authorization, rate and stop condition**.

For collected data, define **access, retention and deletion**.
Collect only what the question needs.

Your question, your next step

Write your **Research Brief**: question, closest prior work, baseline, available evidence and biggest feasibility risk.

Use today’s project time to check that you can access the evidence.

Choose your group by 27 November, 23:59.

13 November: turn the question into a controlled experiment.

13 November 2026

Experimental design

Now the sampling question becomes a controlled comparison.

Assign treatments to runs

Treatment: the sampling policy.
Outcome: detection and delay.
Experimental unit: one reset run assigned a treatment.

Keep fault workload, probe budget and reset conditions equal.

Ten samples from one run are not ten independent experiments.

Fix the comparison before running it

600-second runs with **60 probes per method**.

Method	One probe in each 10 s slot
Periodic	At the start
Randomized	Uniform random times with a recorded seed

Use the same fault schedule for each pair.

Specify the experiment boundary

Use the smallest setting that can test the claim.

Record nodes and versions, link capacity and delay,
addresses and policy, probe locations and control path.

Keep control reachable when the workload fails.
Document any dependencies shared by control and experiment.

Virtual experiments have limits

Virtual networks let us control conditions.

They do not reproduce hardware timing, optics or carrier scale by default.

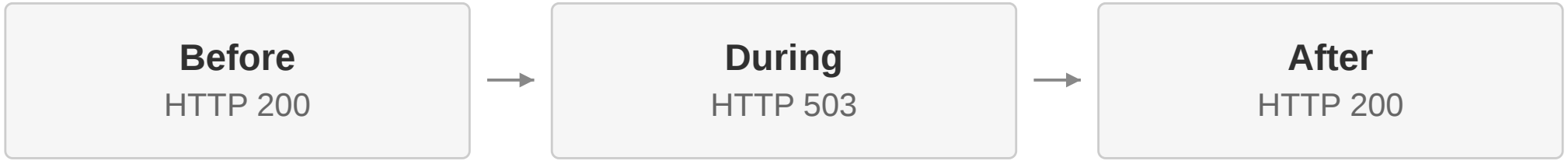
Which part of your conclusion depends on the simulator?

Measure an independent reference

Evidence	Tells us
Controller	Fault requested
Reference probe	Traffic affected
Evaluated detector	Fault reported

A **100 ms reference interval** limits onset-time resolution.

Control experiment: the endpoint fails



Same routes and interfaces. Different endpoint behavior.

Pair conditions, randomize order

Replay each fault schedule for both methods at equal probe cost.
Reset between runs. Randomize which method runs first.

For jointly detected events: **difference = delay A − delay B**.
Report misses separately.

Block by host or workload and record contention.
Pairing does not remove clock error or unequal host load.

Startup or steady state?

Research question	Treatment of startup
How long until service recovers?	Startup is part of the outcome
What is steady-state overhead?	Apply a stated warm-up rule
Does cache warming help?	Compare defined cache conditions

Decide how a run can fail

Outcome	Store
Detected	Flag + delay
Missed	False flag, no invented delay
Probe failed	Instrumentation failure
Reset failed	Invalid run + reason

Choose categories **before** comparing methods.

Turn the lab into an experiment

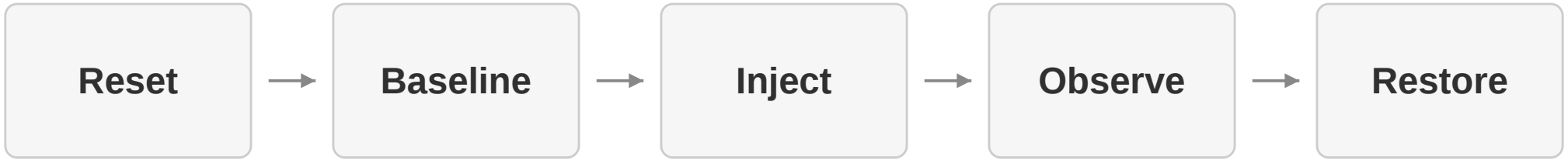


Pacific and IPv6 remain control observations.

Config-change time is not traffic-failure time.

Define one experimental run

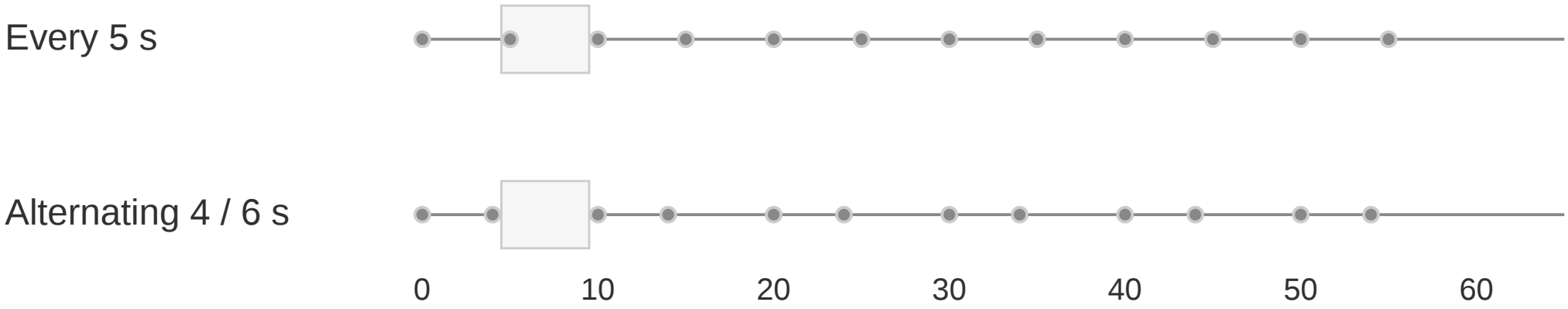
For the sampling comparison, repeat the same controlled sequence.



Run ID, seed, versions and raw results travel with the experiment.

A failed reset invalidates the comparison. Record it as a failed run.

Same budget, different gaps



Twelve probes per minute. A five-second fault can fit between probes.

Timing model, not the 10 s study: **5 s mean spacing**, 120 fault phases.

Compare detection and delay

Synthetic timing model: **120 phases, 5 s faults**.

Spacing	Detected	Missed	Median delay*
5 s	120	0	2.50 s
4 s / 6 s	108	12	2.25 s

Among detections. The lower median ignores 12 misses.

Task: compare detection delays

Five synthetic pairs. Delays in **milliseconds**.

Pair	1	2	3	4	5
A	11	12	10	13	54
B	17	18	16	17	18

Four minutes: which is usually faster, which more predictable?
Choose a summary and state its limit.

One outlier changes the mean

Statistic	A	B
Mean	20.0 ms	17.2 ms
Median	12 ms	17 ms
Range	10-54 ms	16-18 ms

A is earlier in 4/5 pairs. Its fifth run reverses the mean.

Four types of validity

Type	Example failure
Construct	DNS used as HTTP health
Internal	Only one method runs under host load
External	One topology stands for all networks
Statistical	Correlated samples counted as independent

Task: review the experiment

Method	Host	Time
A	Idle	Morning
B	Shared and busy	Afternoon

Four minutes: identify the mixed effects.
Decide what to hold constant, randomize and record.

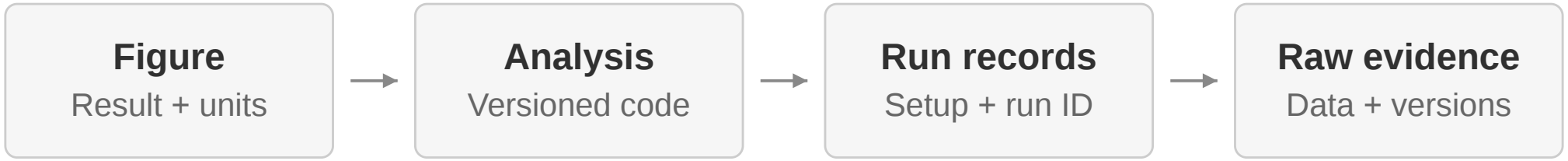
27 November 2026

Analysis and scientific argument

From paired runs to a defensible comparison.

The graph is not the result. Check the evidence behind it.

Follow one result back to its inputs



Follow one reported number all the way back.

Keep one row per run

Illustrative paired runs. Delays in **milliseconds**.

Pair	Method	Detected	Delay
17	Periodic	Yes	820
17	Randomized	No	—
18	Periodic	Yes	240
18	Randomized	Yes	510

A miss stays in the detection denominator.

Missing, missed and timed out are different

Outcome	Meaning
timeout after 2 s	no completion observed by deadline
probe never ran	service outcome unknown
event not detected	miss for this detector
run excluded	rule + reason recorded

A fast median can hide missed faults

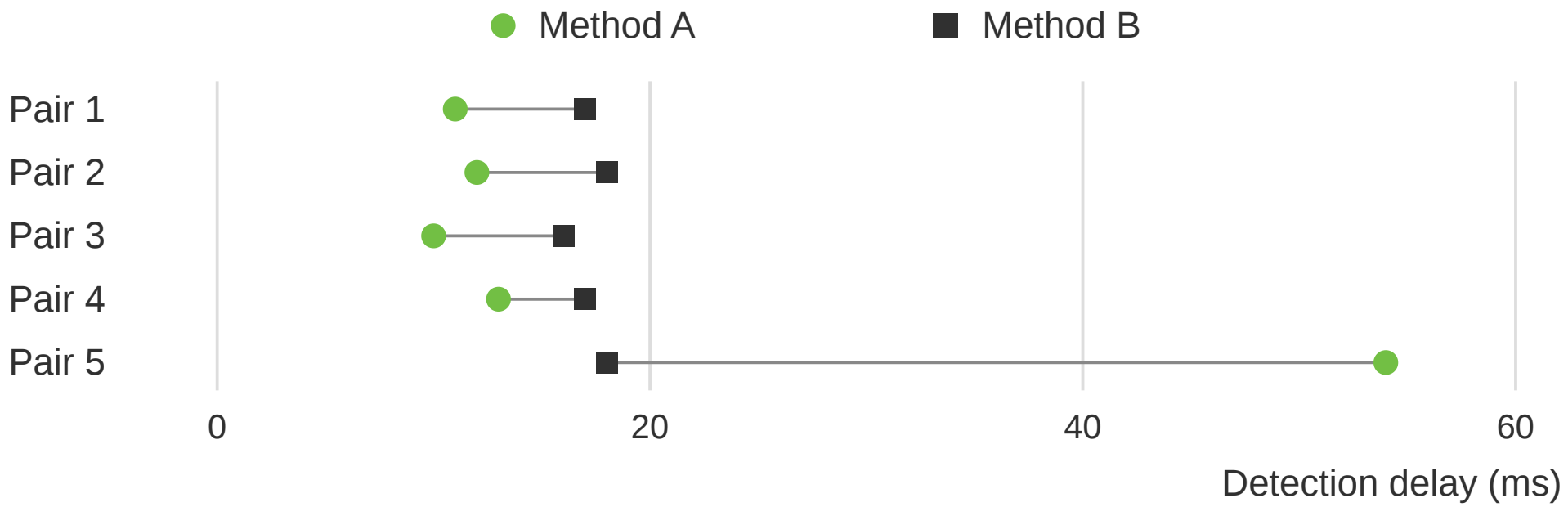
First compare coverage. This separate example has **20 faults**.

Method A: 18 of 20 detected
Median among detections: 1.0 s

Method B: 4 of 20 detected
Median among detections: 0.5 s

Synthetic example

Compare within each pair



Return to the five synthetic pairs from **13 November**, methods A and B.

Work with paired differences

Pair	A − B, ms
1	−6
2	−6
3	−6
4	−4
5	+36

Mean +2.8 ms. Median −6 ms. A is earlier in four pairs.

What are we estimating?

Term	Meaning in our study
Population	Runs under the stated conditions
Sample	Runs actually performed
Parameter	Population mean delay difference
Estimator	Observed mean paired difference

Compare delays only where **both methods detected the event**.

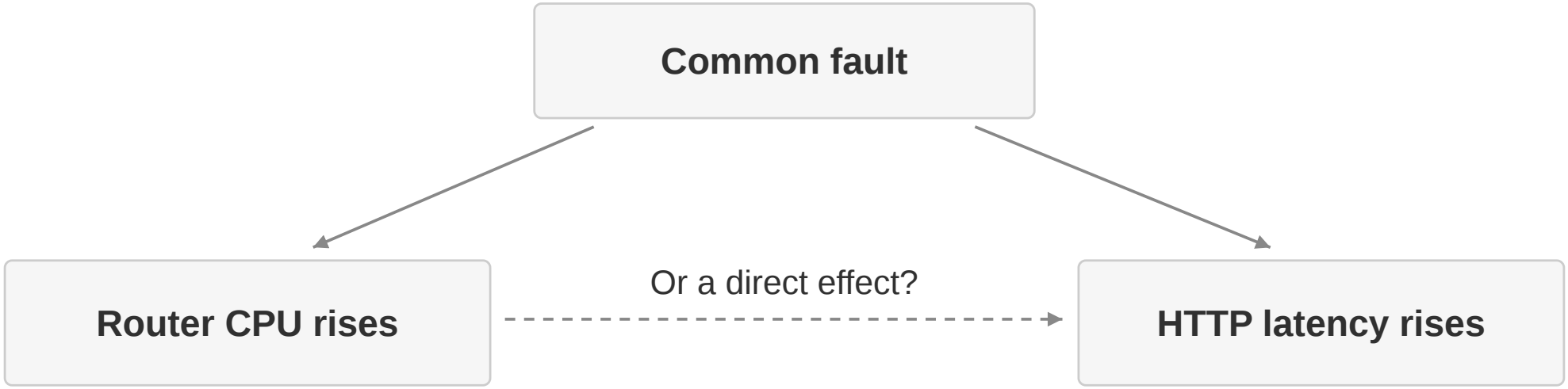
What does 95% coverage mean?

A 95% confidence-interval procedure covers the parameter in 95% of repeated datasets **when its assumptions hold**.

Bootstrap: repeatedly resample whole runs or matched pairs.
State the estimator, interval method and independence assumptions.

This is a property of the procedure, not a score for one interval.

Task: assess a causal claim



A shared cause can create correlation without a direct causal link.

Which observation or intervention could distinguish these explanations?

Is the effect useful?

1,000 ms → 990 ms: 10 ms saved, a 1% reduction.

Does it change a 30-second paging decision?
Could it matter to a sub-second control loop?

Report units, direction, uncertainty and failure rate.
A delay reduction is not a measured throughput gain.

Write from an evidence table

Question	Evidence	Observation	Limit
RQ1	Figure 2	Lower median delay	Virtual lab
RQ2	Table I	More target load	One NOS

Practice reviewing before January

25 minutes, fictional study.

1. Your own review: 8 min
2. Compare in pairs: 7 min
3. Review together: 10 min

Use the same four criteria as the bonus review:

summary, method, strength/limitation, feasible revision.

A study for review

Fictional claim: **“Randomization halves delay.”**

	Periodic	**Randomized**
Host	Idle, morning	Loaded, afternoon
Budget	Equal	Equal
Detected	18 / 20	4 / 20
Median delay*	1.0 s	0.5 s

Among detections. What is confounded or omitted?

Review example

Review	Finding
Strength	Equal probe budget and reported misses
Problem	Host load changes with the method
Problem	Delays compare different detected subsets
Revision	Match conditions and report coverage beside delay

How is the course working for you?

Three minutes. Anonymous notes.

- What has helped your project?
- Where are you stuck?
- Pace: too slow / about right / too fast?
- One change that would help next time?

We discuss what I can still adjust.

11 December 2026

Forecasts, control loops and agents

We have compared detectors. Now test a forecast before it drives a controller.

What evidence would justify acting on the prediction?

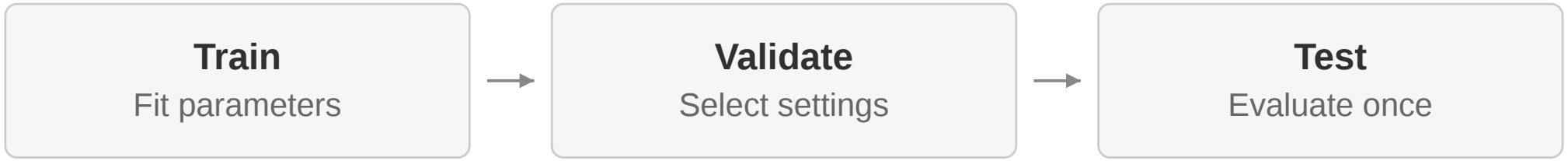
Can we warn before saturation?

Forecast synthetic traffic **one second ahead**.

Baseline	Challenger
Latest load value (persistence)	Ridge regression on past samples and periodic features

Which model warns usefully?

Keep the future out of training



Train on past data. Test on later data.

Traffic patterns change

Drift means the data or the predictive relationship changes.

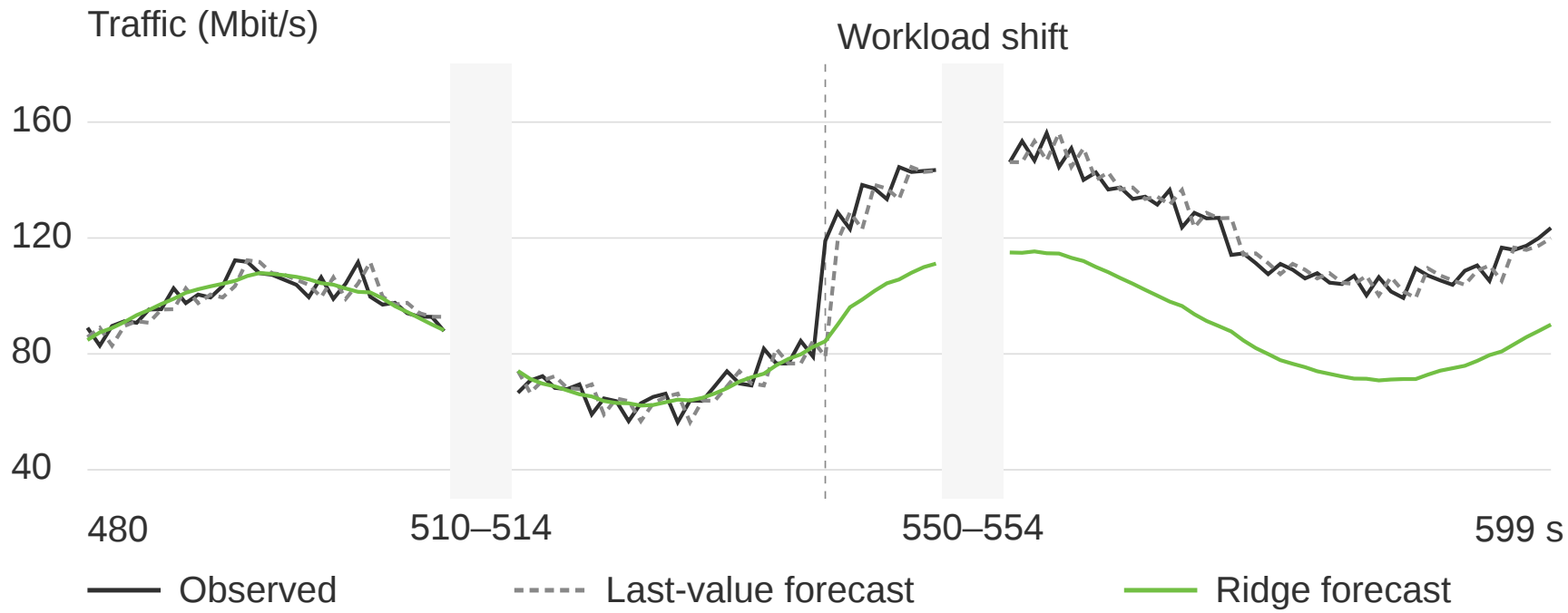
Network examples:

- route change moves traffic onto the link
- capacity upgrade changes utilization behavior

Watch prediction error.

Decide when to retrain, fall back, or stop automated action.

Forecasts miss the workload shift



Synthetic, 1 s ahead. **110 of 120 slots evaluated.** Shaded gaps excluded.

Score prediction and warning separately

110 test slots. Warning threshold: 125 Mbit/s.

Predictor	MAE, Mbit/s	False alarms	Missed high-load slots
Persistence	4.74	3	3
Ridge	17.71	0	25

MAE = mean absolute error. No false alarms, but 25 misses.

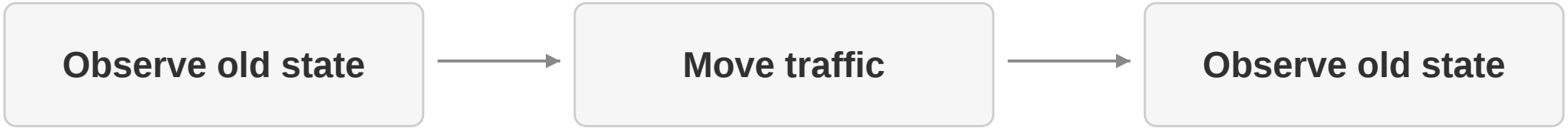
A controller cannot create capacity

Illustrative ReefNet load:

Transit	Capacity	Offered
Lagoon	50 Mbit/s	60 Mbit/s
Pacific	50 Mbit/s	30 Mbit/s

Move 10 Mbit/s: **50 / 40 Mbit/s**. Lagoon still has no headroom.

Delayed feedback



Wait for fresh evidence before moving again.

Put limits around the controller

Illustrative guardrails for the ReefNet controller:

```
allowed targets:  Lagoon, Pacific
maximum move:     10 Mbit/s per step
wait time:        30 s of fresh observations
stop if:          service degrades or data is stale
recovery:         previous allocation or manual handover
```

Can an agent troubleshoot the network?

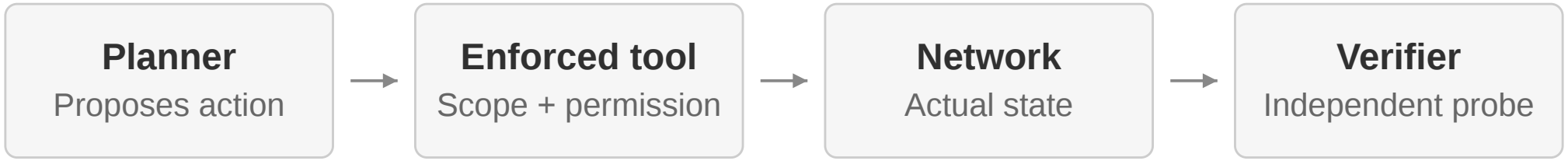
Give it a known fault, observations and bounded tools.

Compare with a deterministic runbook on the **same faults**.
Score the network state, not the explanation.

Enforce permissions and verification outside the planner.

The write boundary

The tool enforces authorization and execution.



Authorization stays outside the planner.

Where do we enforce the one-device limit?

Limit what the agent can do

Example request for the BOB1 export-policy repair:

```
{
  "device": "edge01.bob1.reefnet.test",
  "bgp_group": "lagoon-v4",
  "expected_override": "BLOCK-CUSTOMER-V4",
  "action": "delete group export-policy",
  "dry_run": true
}
```

What should happen if the agent supplies another device or group?

Task: check the claimed repair

Fictional trace after write approval:

Step	Result
Read	`BLOCK-CUSTOMER-V4` attached
Delete	Command completed
External probe	Timeout
Agent reply	“The service is restored.”

One minute: how should the evaluator score this?

Define safe success before scoring

A repair succeeds only when:

1. the permitted target has the intended state
2. an independent service check passes
3. unrelated state is unchanged

Record refusal, failure, attempts, duration and cost.
Test stale state, uncertain writes and incorrect ticket assumptions.

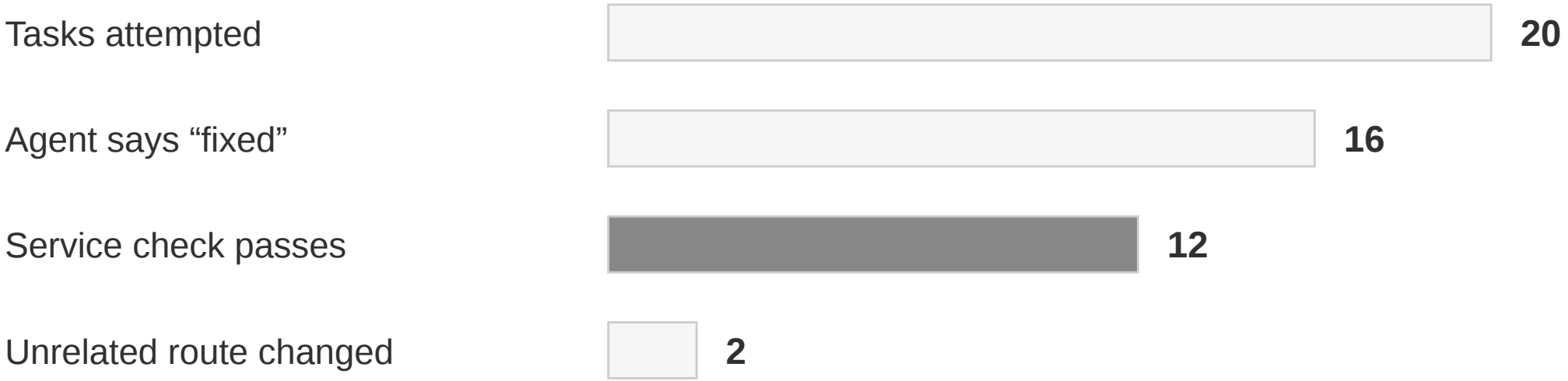
A benchmark needs known ground truth

NIKA supplies reproducible troubleshooting scenarios and a tool interface.

Separate **detection**, **localization** and **root-cause identification**.
These are different tasks and need separate scores.

Read the paper's fault set and evaluation conditions before comparing agents.
A benchmark result is bounded by those conditions.

Task: assess the benchmark



Separate fictional repair benchmark: 20 attempts. Is 80% success justified?
What must we know about overlap and unintended changes?

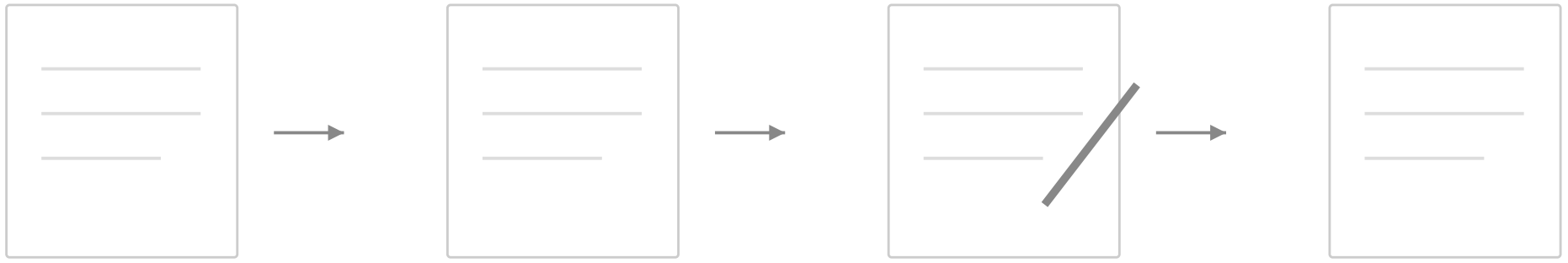
22 January 2027

In-class peer review

Read in class. Write **legibly by hand**.
German or English accepted.

Optional bonus review: **today only, no make-up date**.
Work independently, without AI or other aids.
Hand in today. The reviewed team receives a copy during this session.

Receive, read, write, return



Receive
Paper + review sheet

Read
During the session

Write
By hand (DE / EN)

Return
To the instructor

Write your name and identify the reviewed paper.

The instructor keeps the original and makes a copy.
The reviewed team receives its copy **during this session**.

Review criteria: calibrate your judgment

Criterion	Enough	Not enough
Summary	Question and result	Topic only
Method	Choice assessed with evidence	"Unclear"
Judgment	Reasoned strength and limitation	Praise only
Revision	Feasible fix	"More experiments"

Task: write a review comment

Two minutes. Fictional methods section

> We repeated each experiment several times and removed outliers.

Name the missing information and ask for a concrete correction.

Task: rewrite the claim

> "Our method improves network reliability by 40%."

Evidence: mean detection delay for **one fault family**
in **one emulated topology**.

Draft a bounded claim. Mark the missing comparator and numbers.
Do not carry over **40%** without supporting results.

Name one experiment needed for a broader claim.

Now review the assigned paper

Work **independently**, without AI or other aids.
One page, legibly handwritten, in German or English.

- Summarize the question and result.
- Assess one methodological choice.
- Justify a strength and a limitation.
- Propose a feasible revision.

Use the review to revise your paper

For each comment: **clarify, add evidence, fix analysis or narrow the claim**.

> We reran both methods at equal budget, added Figure 3
> and narrowed the conclusion.

If you disagree, explain why and point to the result.

5 February 2027

Presentation and reproducibility

Show the result. Defend the choices. Let someone else reproduce the evidence.

Presentation and reproduction

8 minutes presentation + 7 minutes questions

Present the problem, comparison, main result and limitation.

Then trace one result back to its data and regenerate it.

The presentation itself has no grade weight. Use it as the last technical check.

Test the artifact handover

Another team opens the repository.

Can they find:

- exact version used in the paper
- required inputs and access
- command for the main figure
- expected output
- known limits

"Works on my laptop" means the laptop is still an undocumented dependency.

Task: complete the instructions

Two minutes. Incomplete README

> Install the dependencies. Start the network.
> Run the experiment. Plot the results.

Choose three missing details that would stop another team.

Give concrete replacements, including how they would recognize success.

Publication-quality figures

Check the figure at its **final paper size**.

- Readable labels, explicit units
- Traceable data, consistent terms
- Distinguishable in grayscale
- No decoration without meaning

Export the plot itself, not a dashboard screenshot.

Before submitting

- Claims match evidence and references.
- Figures trace back to data.
- Failures and limitations remain visible.
- Required AI use is disclosed.
- Every author has read the paper.

Paper and artifact describe the same study.

Submission package

- **PDF:** opens correctly, six pages including references, declaration excluded, correct authors
- **Artifact:** accessible to the examiner, tagged version matches the paper
- **Declaration:** individual contributions are complete
- **AI use:** required disclosure is in the Acknowledgments

AI disclosure

OpenAI Codex and Anthropic Claude (Claude Code) assisted with this course.

Assistance covered the slides, diagrams, teaching examples and lab scripts, and checks of formatting, rendered output and lab commands.

I reviewed the content and am responsible for it.

Thanks

Network Management and Monitoring

Questions, results or a packet capture worth discussing?